

AWS Assignment

AWS CloudFormation

with SNS, SQS, SES, RDS & Route 53

XYZ Corporation – step-by-step implementation with screenshots

Assignment	Topic	Main resources
1	Stack notifications with SNS	s3-cloudformation-stack, SNS topic XYZ-CloudFormation-Notifications, email subscription
2	SQS FIFO queue and SES email	xyz-sqs-fifo-stack (xyz-test-queue.fifo), SES identity sowmeyam2626@gmail.com
3	3-tier web application stack	XYZ-Web-Tier stack: web EC2, app EC2, RDS MySQL (Retain), Route 53 hosted zone

Region: US East (N. Virginia) – us-east-1 for all assignments.

Contents

Assignment 1: CloudFormation Stack Notifications with SNS.....	3
Step 1: Review the stack from CloudFormation task 1.....	3
Step 2: Create the SNS topic.....	4
Step 3: Subscribe your email and confirm it.....	4
Step 4: Attach the SNS topic to the stack.....	6
Step 5: Check the stack events and email notifications.....	7
Assignment 2: SQS FIFO Queue and SES Email.....	9
Step 1: Write the SQS FIFO template.....	9
Step 2: Create the SQS stack.....	9
Step 3: Send a test message.....	11
Step 4: Receive the message.....	12
Step 5: Register your email address in SES.....	12
Step 6: Confirm the identity is verified.....	15
Step 7: Send a test email to yourself.....	15
Step 8: Review the stacks.....	17
Assignment 3: 3-Tier Web Application with CloudFormation.....	18
Step 1: Build the web tier template.....	18
Step 2: Create the XYZ-Web-Tier stack.....	19
Step 3: Add the application tier.....	21
Step 4: Add the database tier with RDS MySQL.....	23
Step 5: Security group rules for each tier.....	26
Step 6: Set up the Route 53 hosted zone.....	27
Step 7: Proposed solution for the development team.....	28
Conclusion.....	29

Assignment 1: CloudFormation Stack Notifications with SNS

Problem statement: XYZ Corporation's team needs to deploy a similar architecture many times for testing, development and production. Implement CloudFormation for the tasks below.

Tasks:

1. Use the template from CloudFormation task 1.
2. Add notifications to the CloudFormation stack using SNS so that you get an email for every step of the stack process.

Step 1: Review the stack from CloudFormation task 1

The existing stack **s3-cloudformation-stack** creates a versioned S3 bucket. Its template is:

```
AWSTemplateFormatVersion: '2010-09-09'
Resources:
  S3Bucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: intellipaat-sowmeya-20260928
      VersioningConfiguration:
        Status: Enabled
```

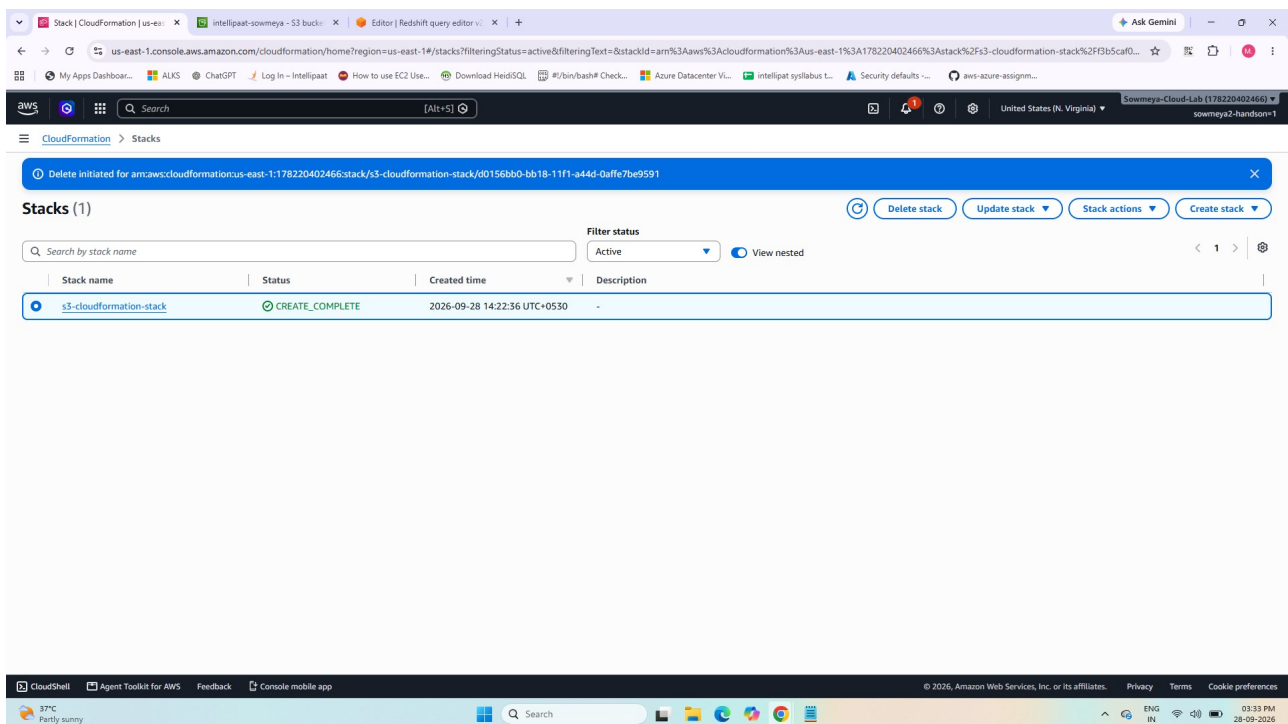


Figure 1: s3-cloudformation-stack in CREATE_COMPLETE state

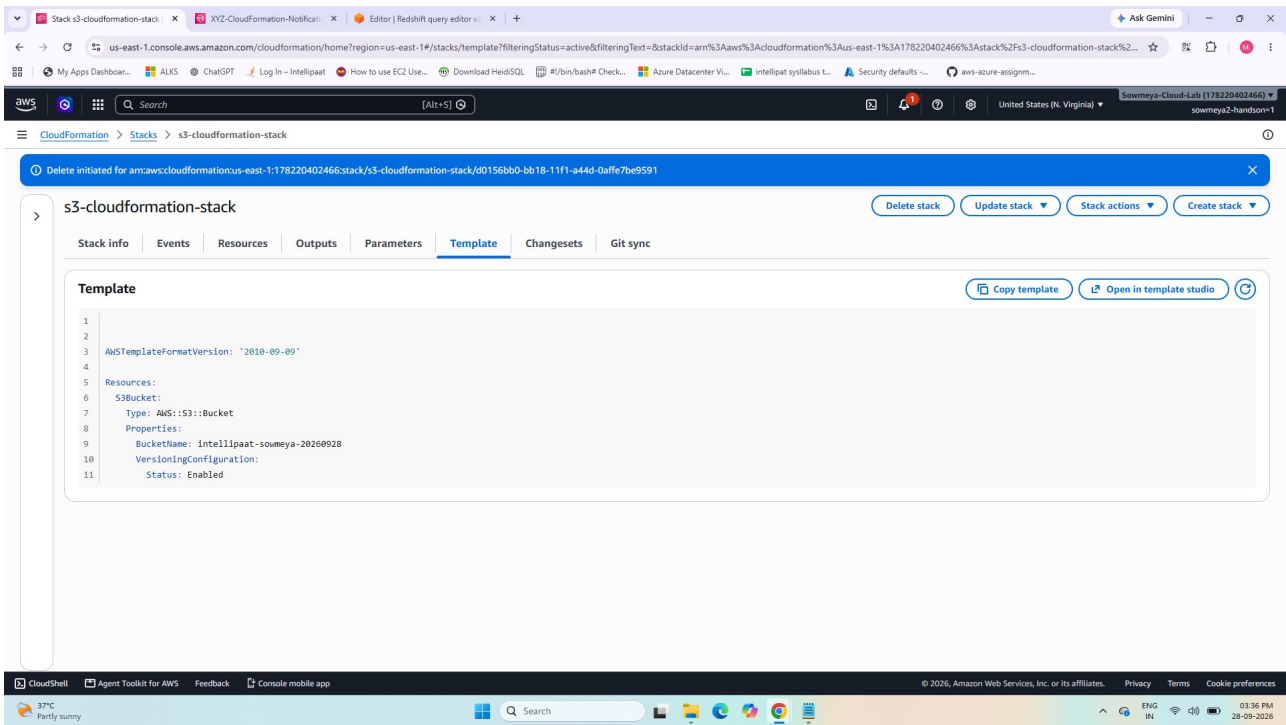


Figure 2: Template tab – S3 bucket with versioning enabled

Step 2: Create the SNS topic

1. Open **Amazon SNS** → **Topics** → **Create topic**.
2. Type: **Standard**. Name: **XYZ-CloudFormation-Notifications**. Click **Create topic**.

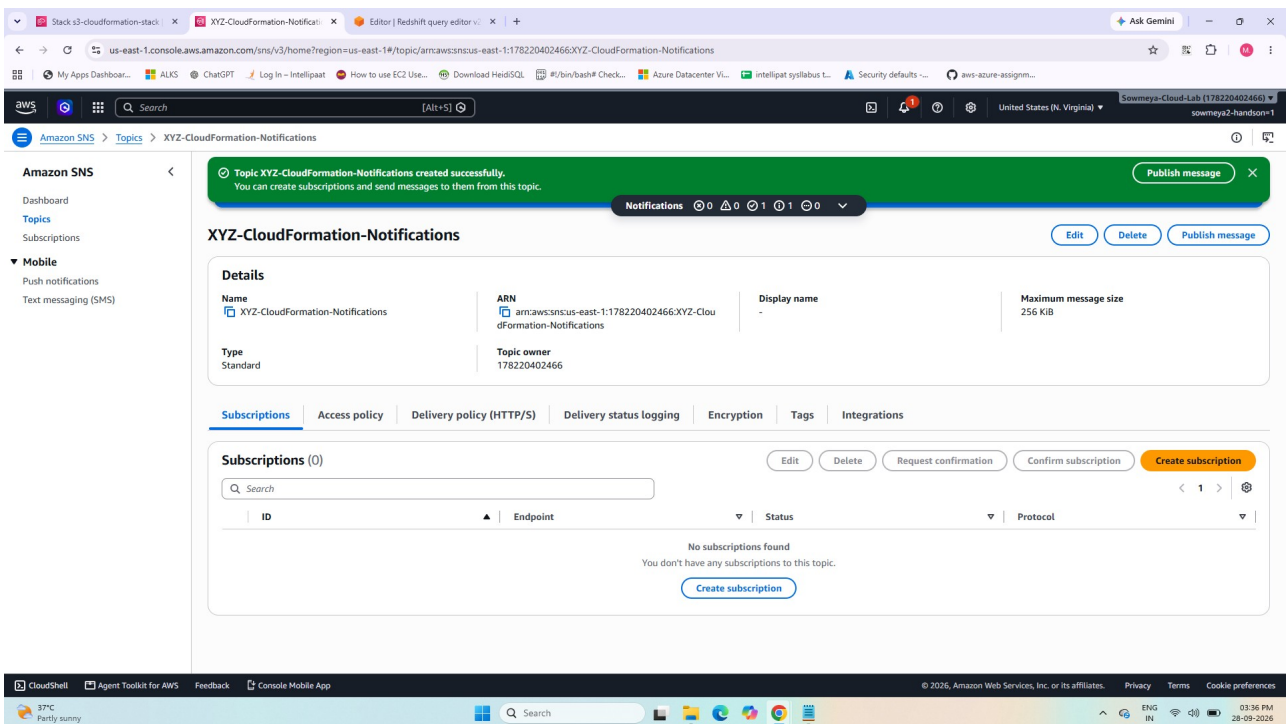


Figure 3: SNS topic XYZ-CloudFormation-Notifications created (Standard)

Step 3: Subscribe your email and confirm it

1. In the topic, click **Create subscription**. Protocol: **Email**. Endpoint: **sowmeyam2626@gmail.com**.
2. Click **Create subscription**, open the **AWS Notification – Subscription Confirmation** email and click **Confirm subscription**.

Create subscription

Details

Topic ARN
arn:aws:sns:us-east-1:178220402466:XYZ-CloudFormation-Notifications

Protocol
Email

Endpoint
sowmeyam2626@gmail.com

After your subscription is created, you must confirm it. [Info](#)

Subscription filter policy - optional [Info](#)
This policy filters the messages that a subscriber receives.

Redrive policy (dead-letter queue) - optional [Info](#)
Send undeliverable messages to a dead-letter queue.

[Cancel](#) [Create subscription](#)

Figure 4: Create subscription – Email protocol with the endpoint address

Subscription confirmed!

You have successfully subscribed.

Your subscription's ID is:
arn:aws:sns:us-east-1:178220402466:XYZ-CloudFormation-Notifications:2ec64c3e-9387-41ae-a873-2526a134c7b0

If it was not your intention to subscribe, [click here to unsubscribe](#).

Figure 5: Subscription confirmed page

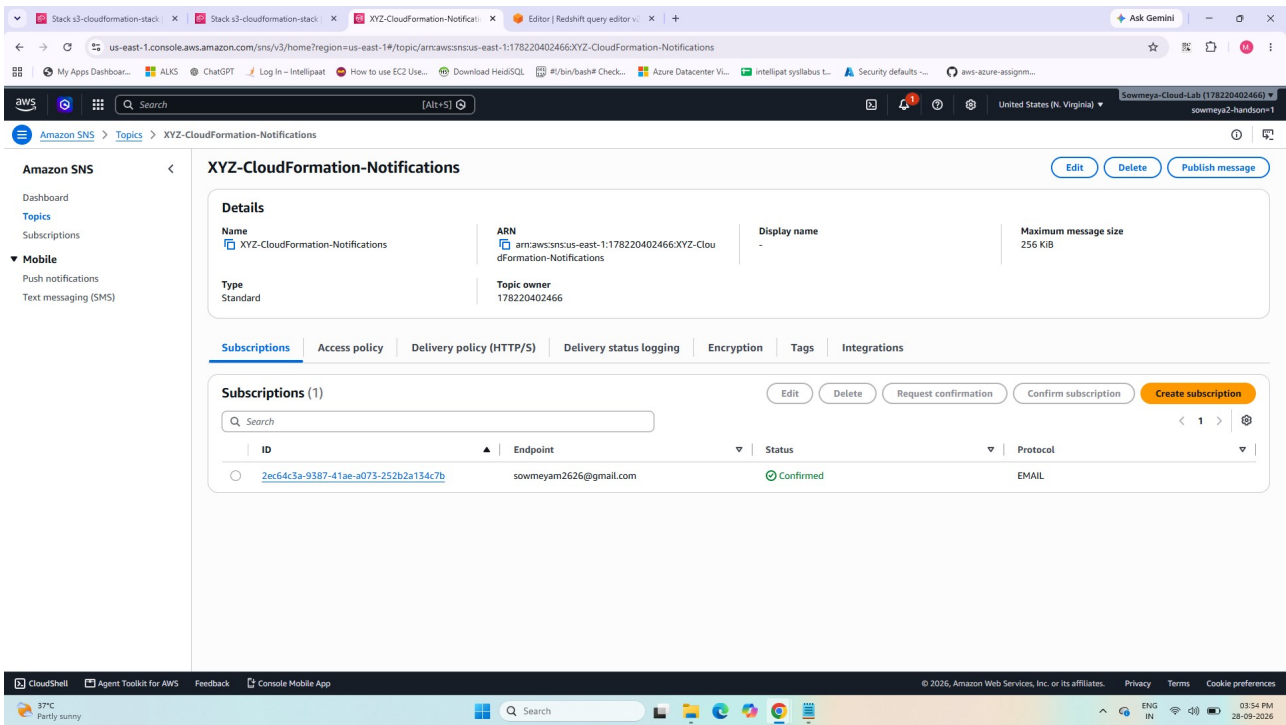


Figure 6: Subscription status Confirmed for sowmeyam2626@gmail.com

Step 4: Attach the SNS topic to the stack

1. In CloudFormation select **s3-cloudformation-stack** → **Update stack** → **Make a direct update**.
2. Choose **Use existing template** and click **Next** until **Configure stack options**.
3. Under **Notification options**, choose **Choose an existing SNS topic** → **XYZ-CloudFormation-Notifications**.
4. Click **Next** → **Submit**.

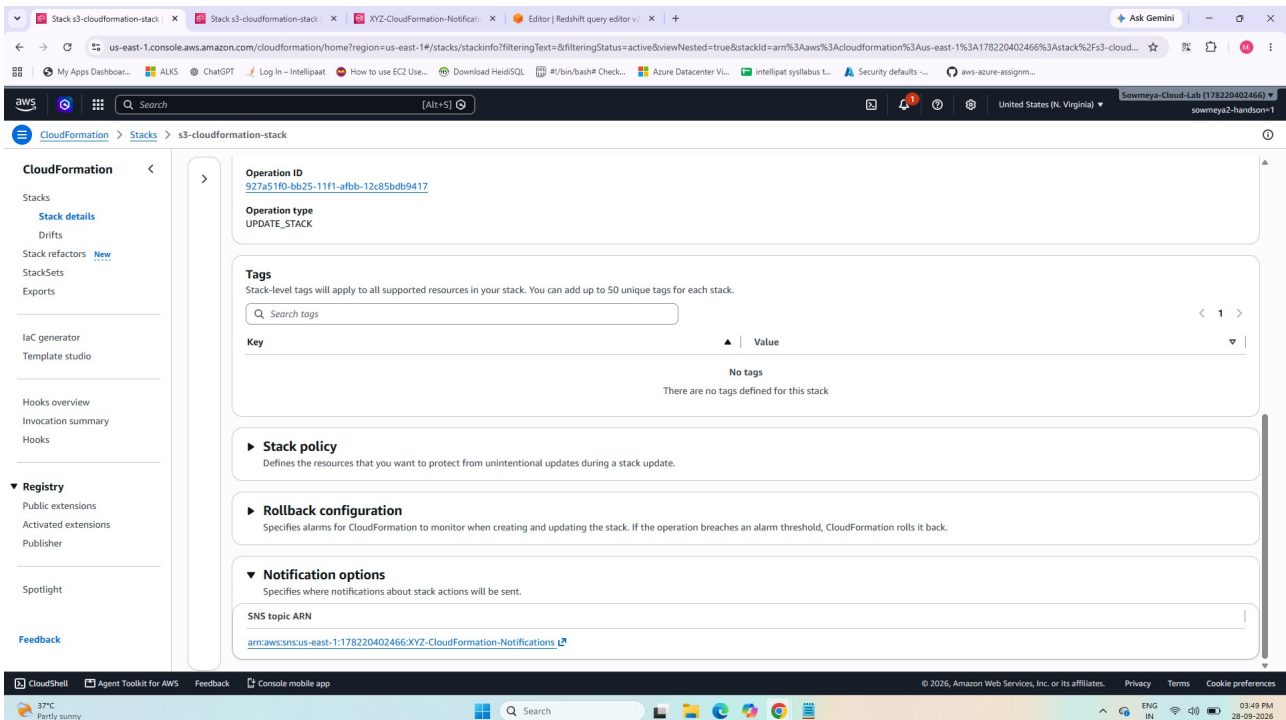


Figure 7: Stack info – Notification options now show the SNS topic ARN

Tip: For a brand-new stack, set the same SNS topic on the **Configure stack options** page during **Create stack** so every CREATE event is emailed.

Step 5: Check the stack events and email notifications

Each stack event (stack and resource status changes) is published to the topic and arrives as a separate email:

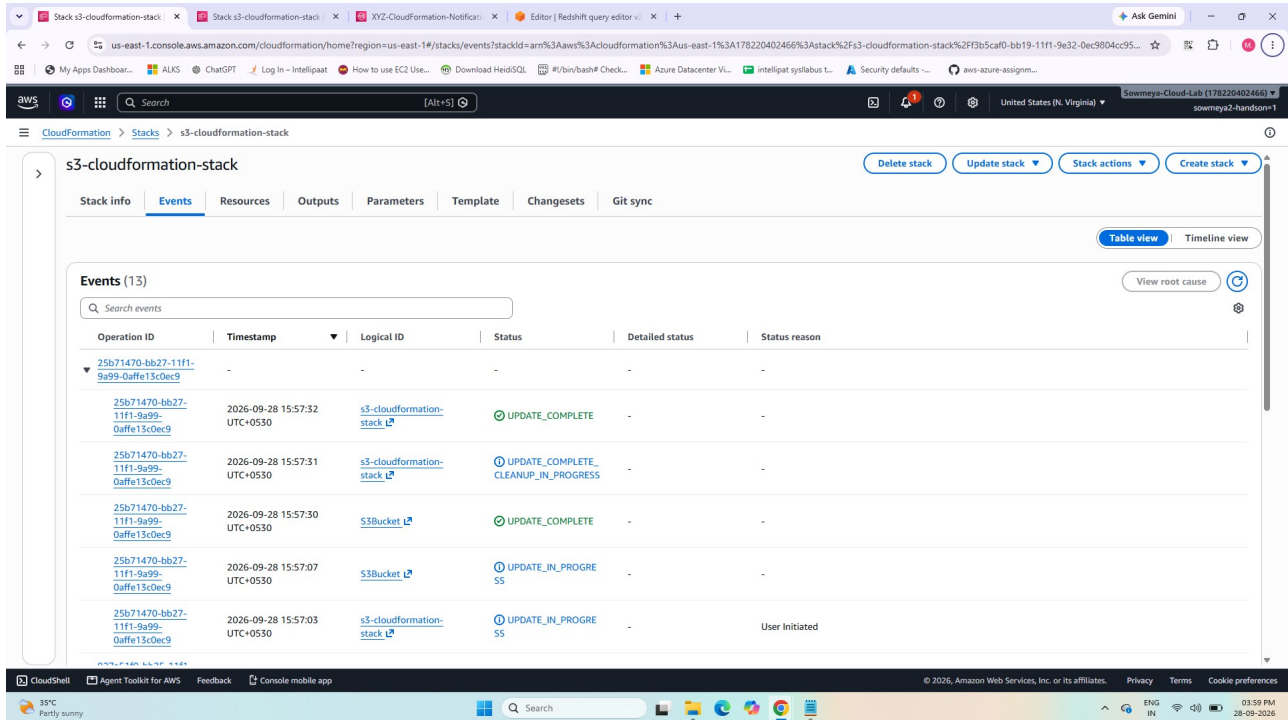


Figure 8: Stack events – UPDATE_IN_PROGRESS through UPDATE_COMPLETE

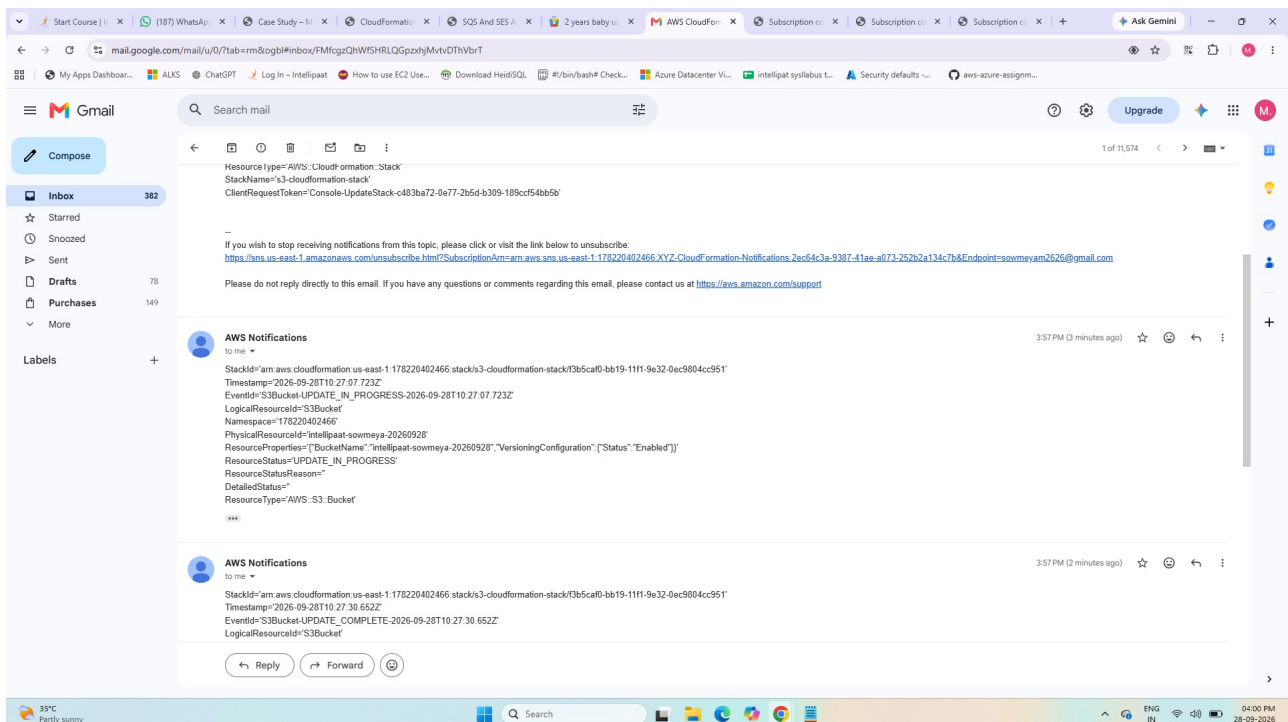


Figure 9: Email notification – S3Bucket UPDATE_IN_PROGRESS

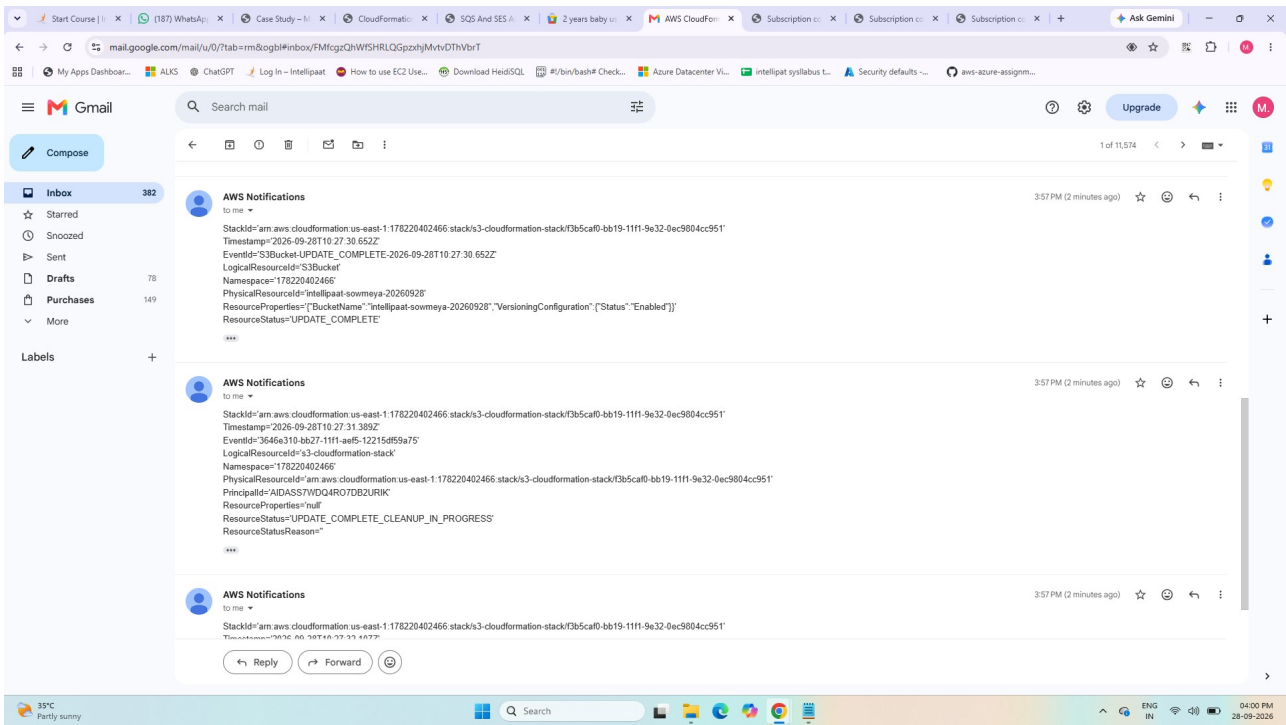


Figure 10: Email notifications – S3Bucket UPDATE_COMPLETE and stack UPDATE_COMPLETE_CLEANUP_IN_PROGRESS

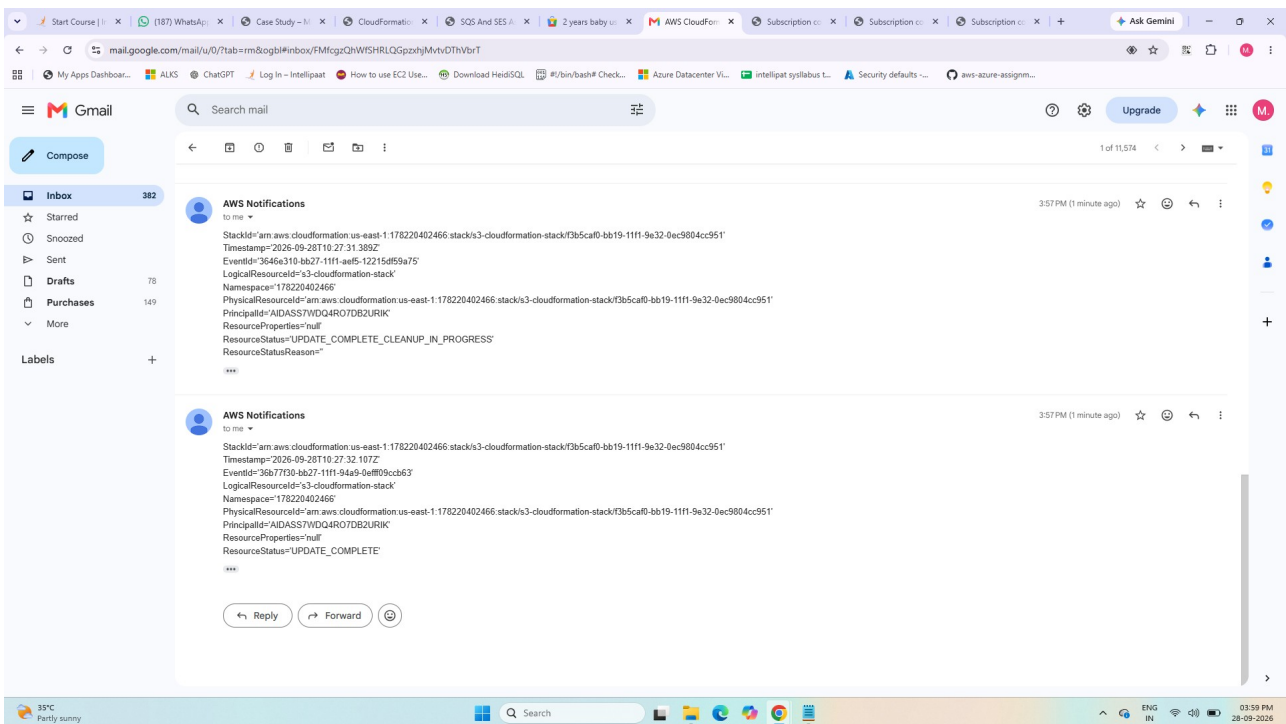


Figure 11: Email notifications – final stack UPDATE_COMPLETE event

Assignment 2: SQS FIFO Queue and SES Email

Problem statement: XYZ Corporation's team needs to deploy a similar architecture many times for testing, development and production. Implement CloudFormation for the tasks below.

Tasks:

1. Create a FIFO SQS queue and test it by sending messages.
2. Register your email in SES and send a test email to yourself.

Step 1: Write the SQS FIFO template

Save the following as **sqs-fifo.yaml**:

```
AWSTemplateFormatVersion: '2010-09-09'
Description: Create a FIFO SQS queue for XYZ Corporation
Resources:
  XYZFifoQueue:
    Type: AWS::SQS::Queue
    Properties:
      QueueName: xyz-test-queue.fifo
      FifoQueue: true
      ContentBasedDeduplication: true
```

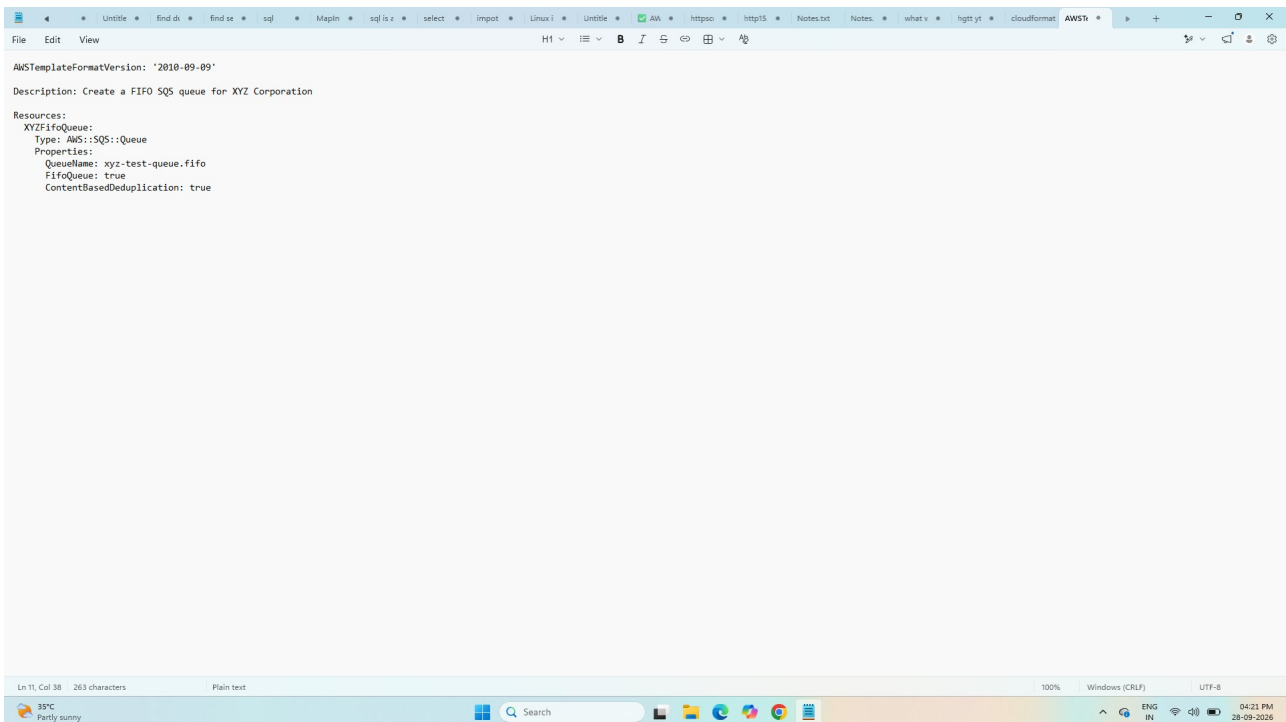


Figure 12: sqs-fifo.yaml template

Note: A FIFO queue name must end with **.fifo**. **ContentBasedDeduplication** lets SQS use a hash of the message body as the deduplication ID, so you don't have to supply one.

Step 2: Create the SQS stack

1. **CloudFormation** → **Create stack** → **With new resources**. Choose **Upload a template file** and select **sqs-fifo.yaml**.
2. Stack name: **xyz-sqs-fifo-stack**. Click **Next** through the options and **Submit**.

Create stack

Prerequisite – Prepare template
You can also create a template by scanning your existing resources in the [IaC generator](#).

Prepare template
Every stack is based on a template. A template is a JSON or YAML file that contains configuration information about the AWS resources you want to include in the stack.

☒ **Choose an existing template**
Upload or choose an existing template.

☐ **Build with template studio**
Build a new template on a visual canvas without leaving the console.

Specify template Info
This [GitHub repository](#) contains sample CloudFormation templates that can help you get started on new infrastructure projects. [Learn more](#)

Template source
Selecting a template generates an Amazon S3 URL where it will be stored. A template is a JSON or YAML file that describes your stack's resources and properties.

☐ **Amazon S3 URL**
Provide an Amazon S3 URL to your template.

☒ **Upload a template file**
Upload your template directly to the console.

☐ **Sync from Git**
Sync a template from your Git repository.

Upload a template file

[Choose file](#)

sqs-fifo.yaml

JSON or YAML formatted file

S3 URL: <https://s3.us-east-1.amazonaws.com/cf-templates-q03ek7brxgak-us-east-1/2026-09-28T105424.5242wch-sqs-fifo.yaml>

[Open in template studio](#)

[Cancel](#) [Next](#)

Figure 13: Create stack – uploading sqs-fifo.yaml

xyz-sqs-fifo-stack

[Delete stack](#) [Update stack](#) [Stack actions](#) [Create stack](#)

[Stack info](#) [Events](#) [Resources](#) [Outputs](#) [Parameters](#) [Template](#) [Changesets](#) [Git sync](#)

[Table view](#) [Timeline view](#)

[View root cause](#)

Events (7)

Search events

Operation ID	Timestamp	Logical ID	Status	Detailed status	Status reason
2a4663c0-bb2b-11f1-8375-12ffc042ff79	-	-	-	-	-
2a4663c0-bb2b-11f1-8375-12ffc042ff79	2026-09-28 16:26:55 UTC+0530	xyz-sqs-fifo-stack	CREATE_COMPLETE	-	-
2a4663c0-bb2b-11f1-8375-12ffc042ff79	2026-09-28 16:26:54 UTC+0530	XYZFifoQueue	CREATE_COMPLETE	-	-
2a4663c0-bb2b-11f1-8375-12ffc042ff79	2026-09-28 16:25:53 UTC+0530	xyz-sqs-fifo-stack	CREATE_IN_PROGRESS	CONFIGURATION_COMPLETE	Eventual consistency check initiated
2a4663c0-bb2b-11f1-8375-12ffc042ff79	2026-09-28 16:25:53 UTC+0530	XYZFifoQueue	CREATE_IN_PROGRESS	CONFIGURATION_COMPLETE	Eventual consistency check initiated
2a4663c0-bb2b-11f1-8375-12ffc042ff79	2026-09-28 16:25:53 UTC+0530	XYZFifoQueue	CREATE_IN_PROGRESS	-	Resource creation Initiated

Figure 14: xyz-sqs-fifo-stack events – XYZFifoQueue and stack CREATE_COMPLETE

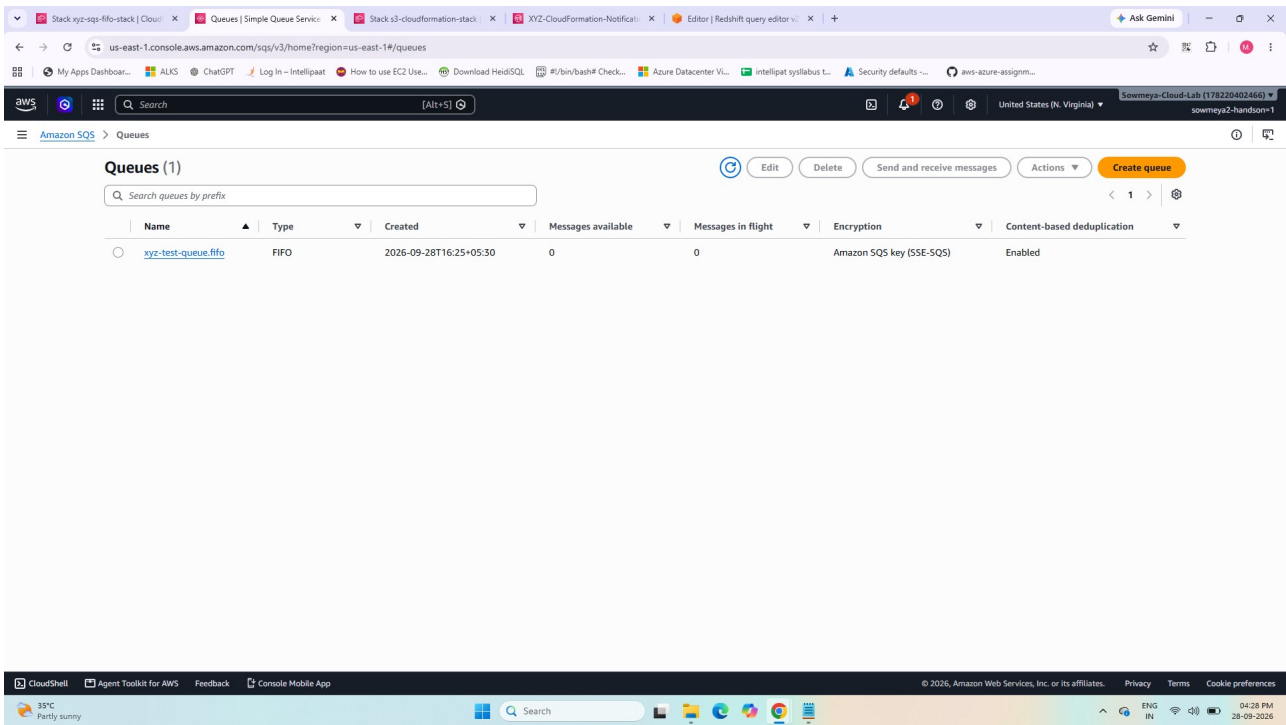


Figure 15: SQS console – xyz-test-queue.fifo (FIFO, content-based deduplication Enabled)

Step 3: Send a test message

1. Select **xyz-test-queue.fifo** → **Send and receive messages**.
2. Message body: **Hello from XYZ Corporation - FIFO SQS test**. Message group ID: **test-group**.
3. Leave the deduplication ID empty and click **Send message**.

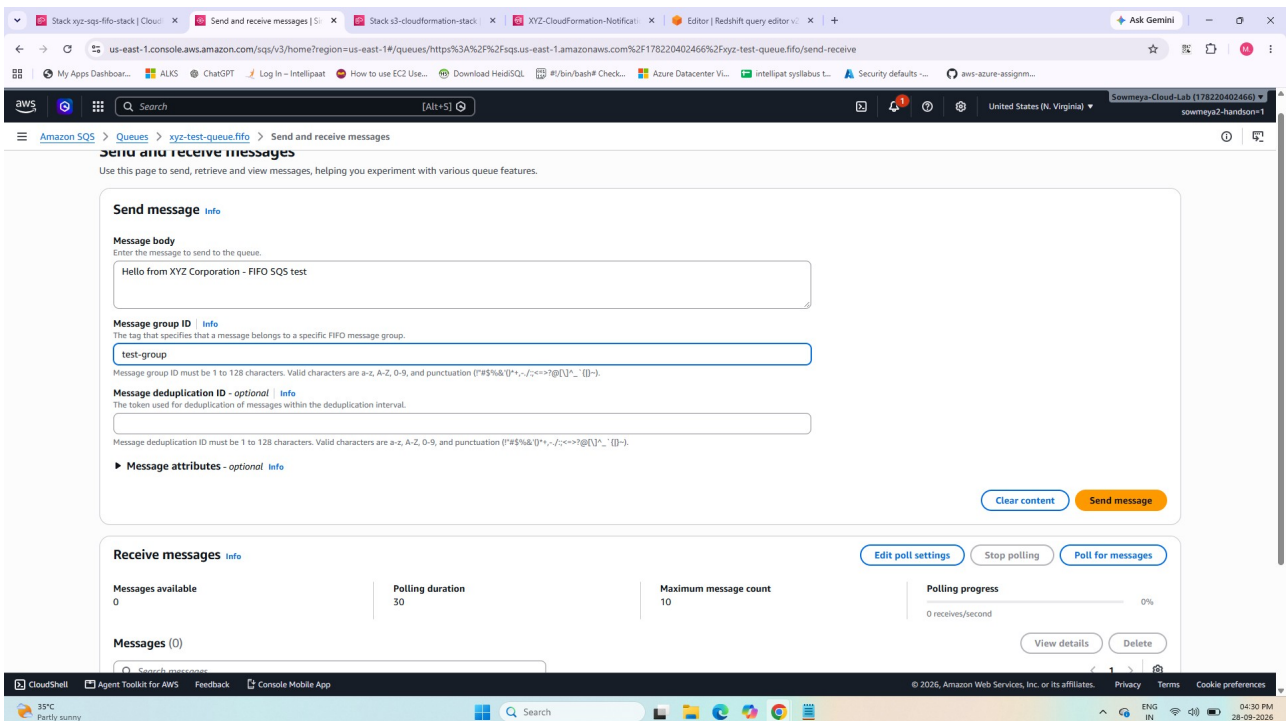


Figure 16: Send message – body and message group ID

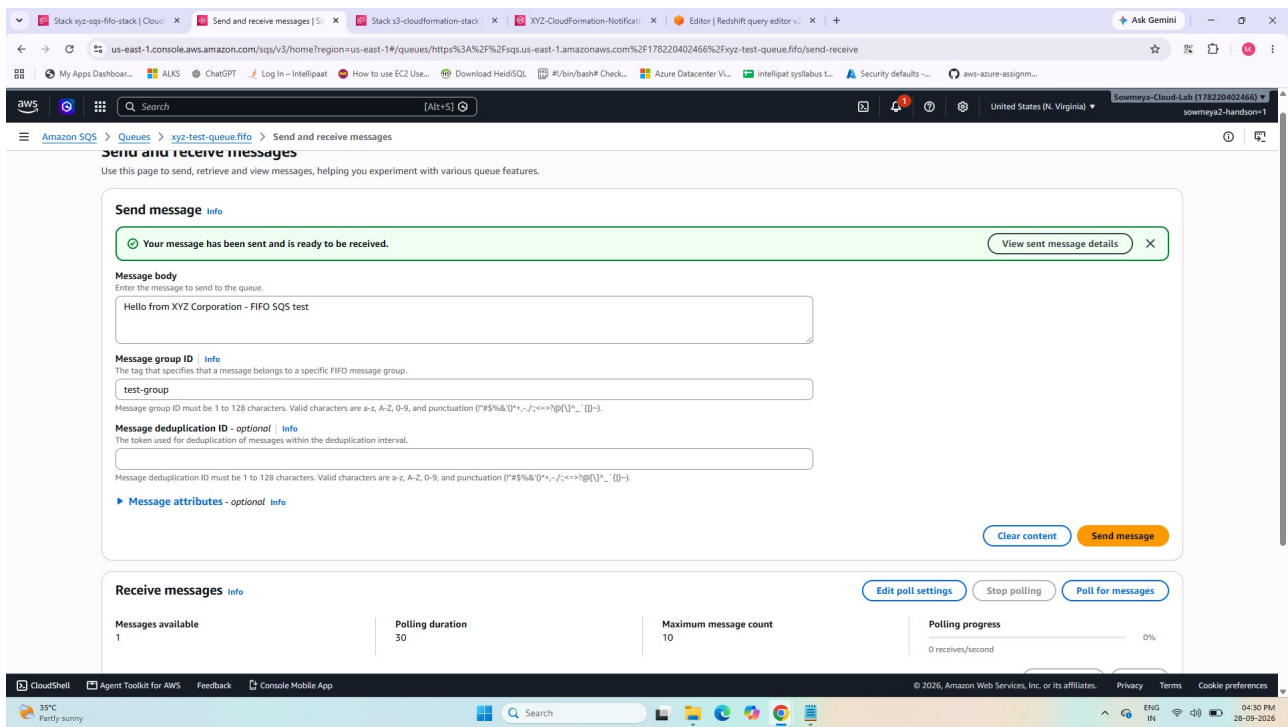


Figure 17: Message sent – Messages available: 1

Step 4: Receive the message

1. Click **Poll for messages** and open the message from the **Messages** list.

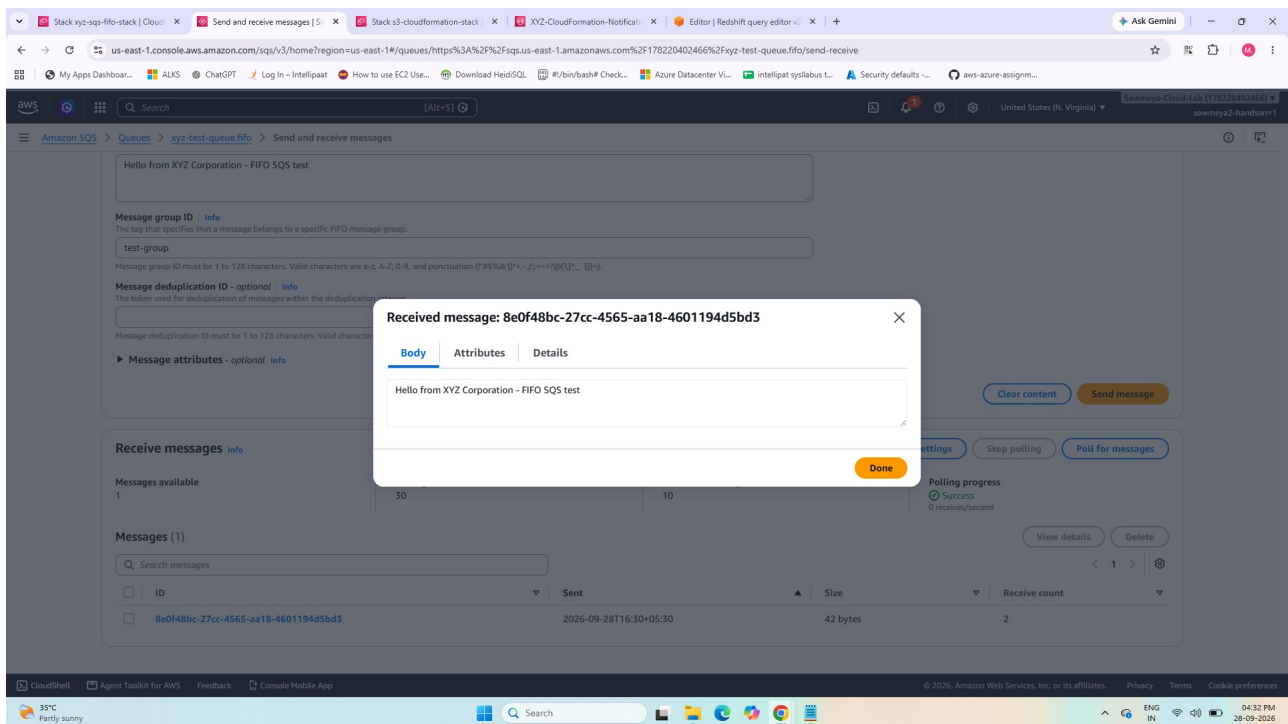


Figure 18: Received message – body matches the message that was sent

Step 5: Register your email address in SES

1. Open **Amazon SES** → **Configuration: Identities** → **Create identity**.
2. Identity type: **Email address**. Email: **sowmeyam2626@gmail.com**. Click **Create identity**.
3. The status is **Verification pending** until you click the link in the verification email (use **Resend** if needed).

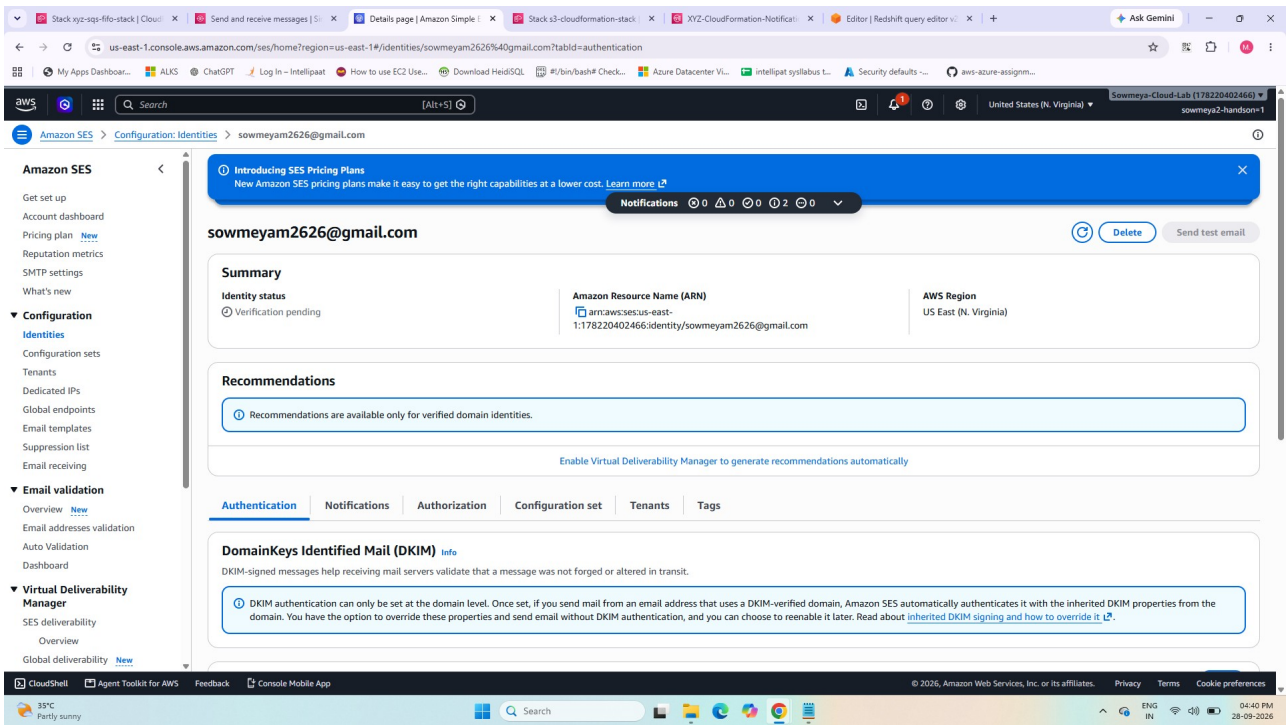


Figure 19: SES identity created – Verification pending

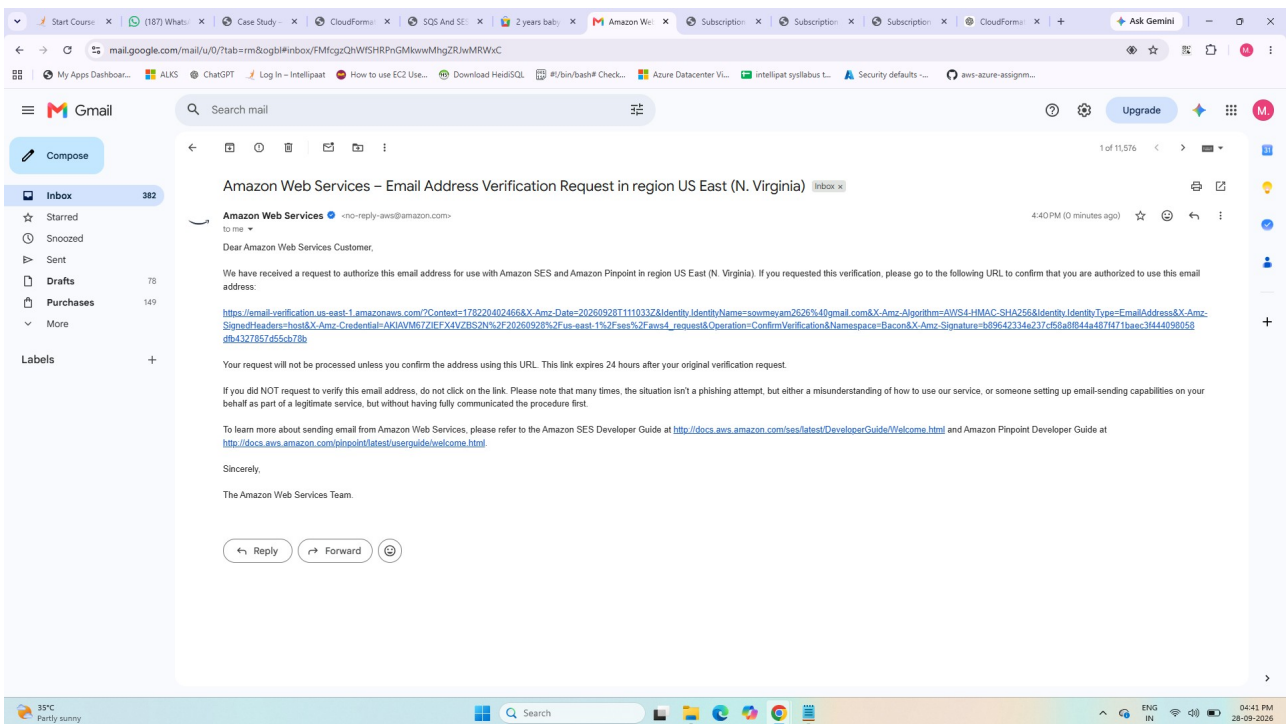


Figure 20: Email Address Verification Request from Amazon Web Services

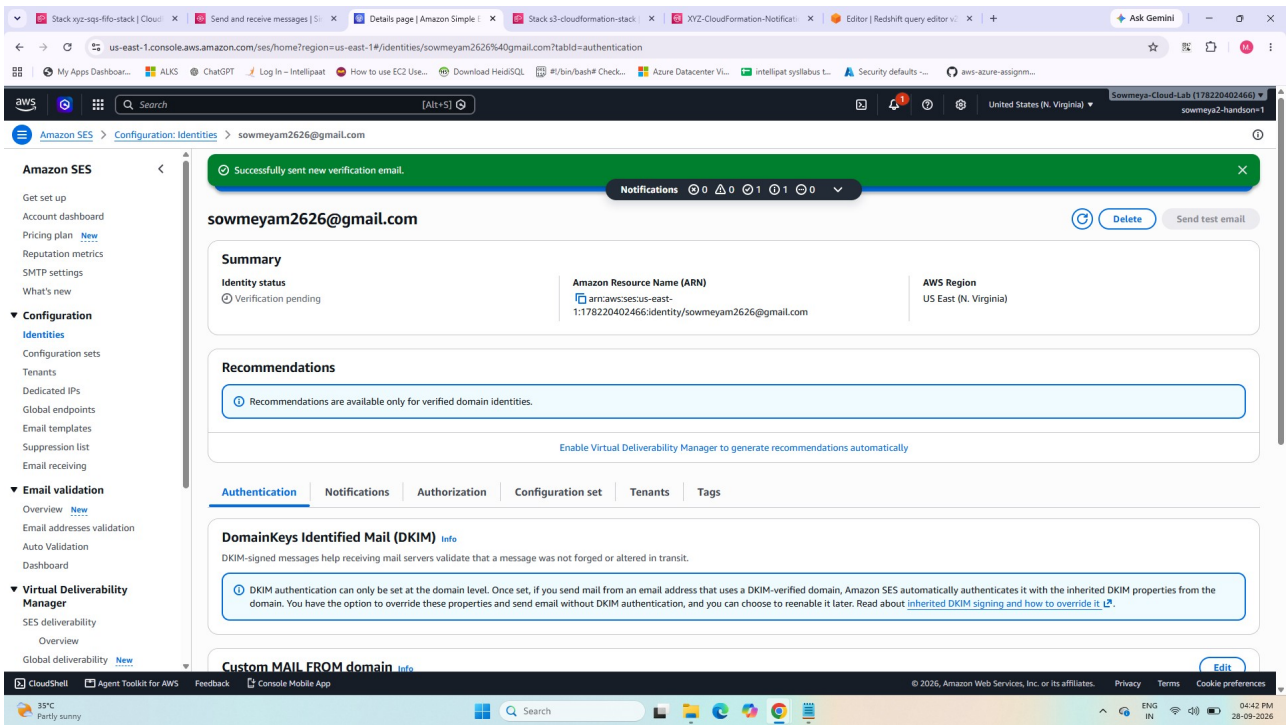


Figure 21: New verification email sent

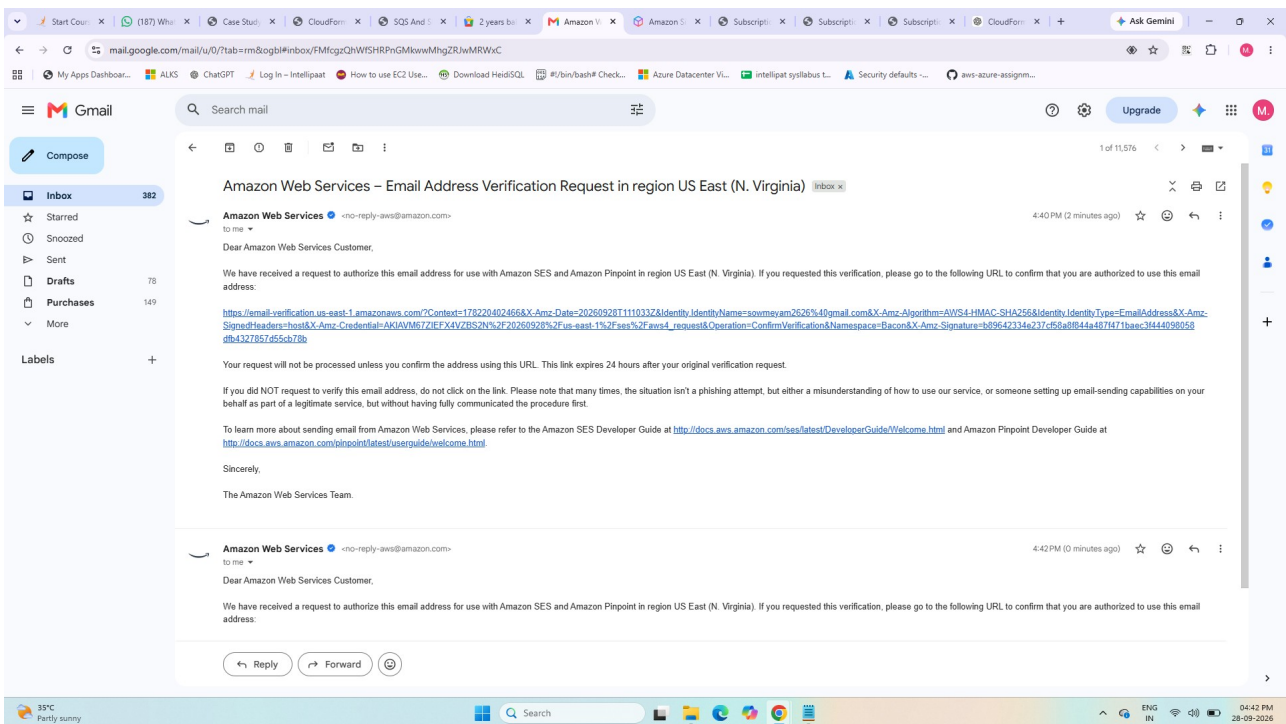


Figure 22: Verification emails in the inbox – click the link to confirm

Step 6: Confirm the identity is verified

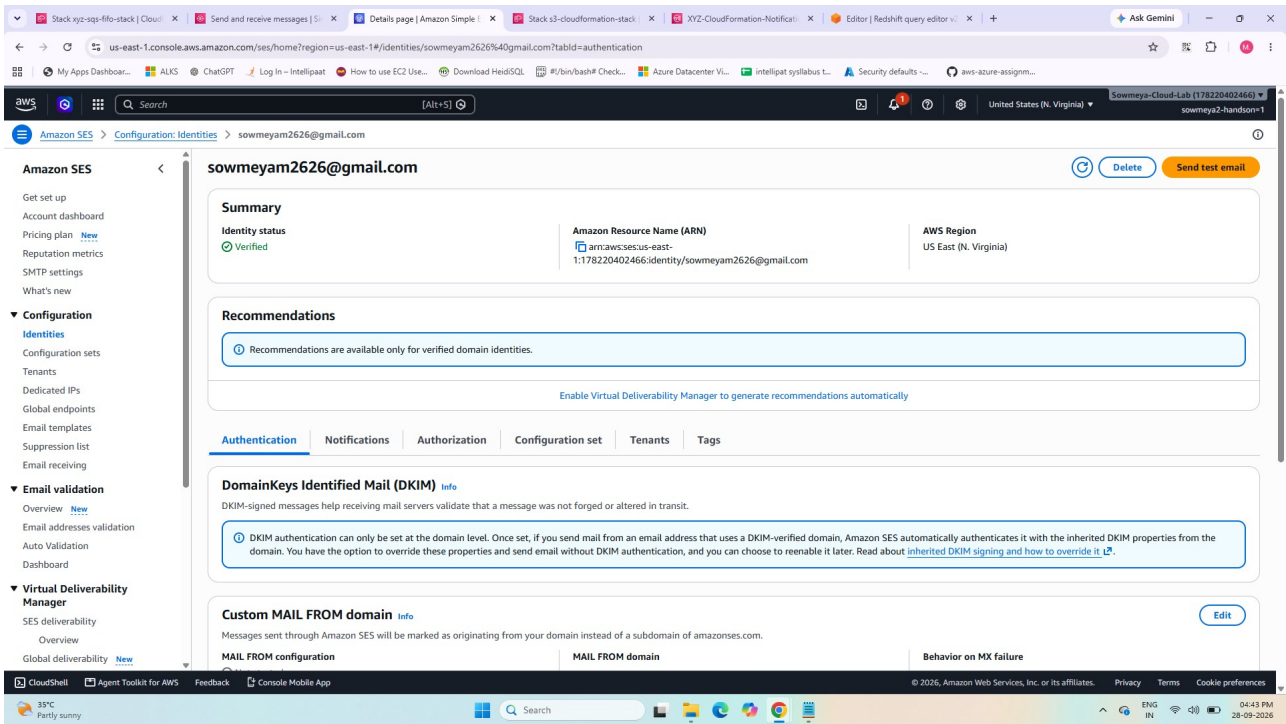


Figure 23: Identity status Verified – Send test email is now enabled

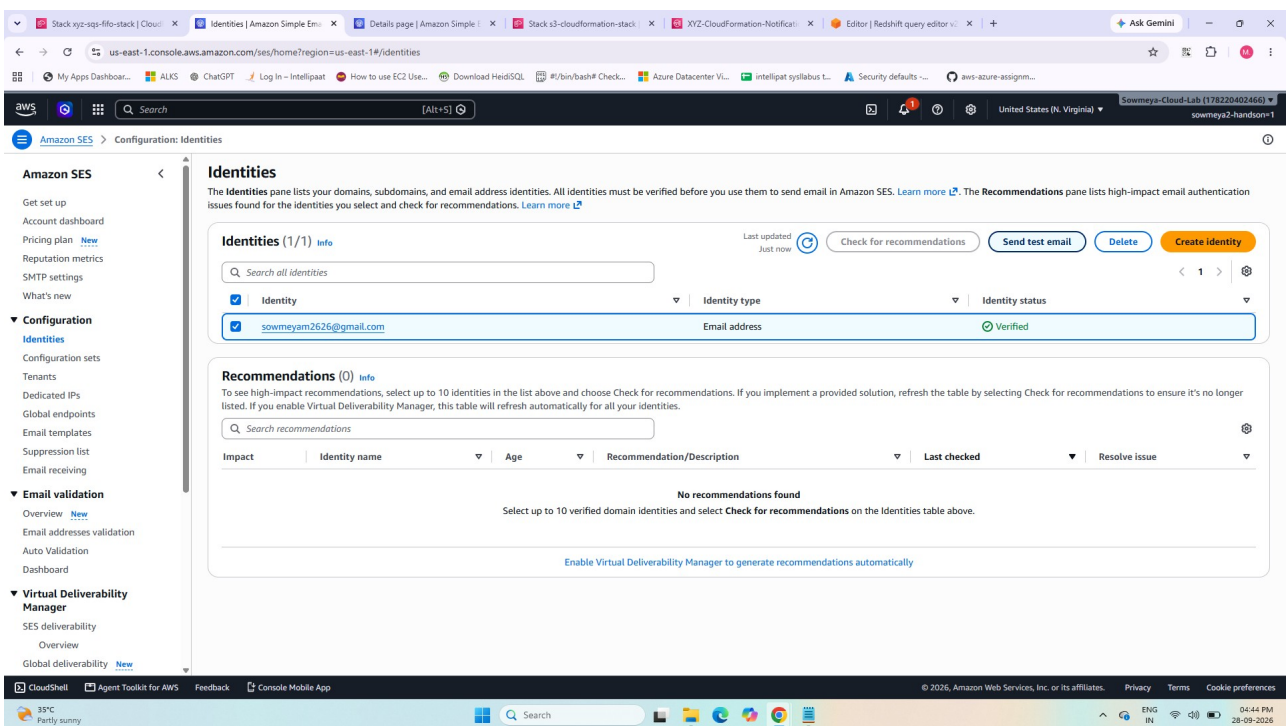


Figure 24: Identities list – sowmeyam2626@gmail.com Verified

Step 7: Send a test email to yourself

1. Select the identity and click **Send test email**. Email format: **Formatted**.
2. Scenario: **Custom** and enter your own verified address as the recipient.
3. Subject: **XYZ Corporation - SES Test**. Add a short body and click **Send test email**.

Send test email info

The Amazon SES mailbox simulator lets you test how your application handles different email sending scenarios. Emails that you send to the mailbox simulator do not count towards your sending quota or your bounce and complaint rates. [Learn more](#)

Message details

Email format

☒ **Formatted**
Choose this option if you want to construct a simple test message using the form provided. SES takes the information entered in the form and parses it into email format for you.

☐ **Raw**
Choose this option if you want to send a more complex test message, such as one that uses HTML or includes attachments. This option requires you to format the entire message yourself.

From-address

sowmeyam2626@gmail.com

Scenario info
Choose the email sending scenario that you want to simulate. Each scenario corresponds to a different recipient email address managed by the mailbox simulator. To specify a custom recipient, select Custom.

Choose a scenario

Subject

XYZ Corporation - SES Test

Body - optional

Hello,
This is a test email sent using Amazon SES for the XYZ Corporation CloudFormation assignment.

Configuration set - optional

Choose a configuration set

► **Additional configurations - optional**

Figure 25: Send test email – from address, subject and body

Amazon SES

Get set up
Account dashboard
Pricing plan
Reputation metrics
SMTP settings
What's new

Configuration

Identities
Configuration sets
Tenants
Dedicated IPs
Global endpoints
Email templates
Suppression list
Email receiving

Email validation

Overview
Email addresses validation
Auto Validation
Dashboard

Virtual Deliverability Manager

SES deliverability
Overview
Global deliverability

Identities

The **Identities** pane lists your domains, subdomains, and email address identities. All identities must be verified before you use them to send email in Amazon SES. [Learn more](#). The **Recommendations** pane lists high-impact email authentication issues found for the identities you select and check for recommendations. [Learn more](#)

Identities (1) info
Last updated 1 minute ago
[Check for recommendations](#) [Send test email](#) [Delete](#) [Create identity](#)

Search all identities

Identity	Identity type	Identity status
sowmeyam2626@gmail.com	Email address	Verified

Recommendations (0) info
To see high-impact recommendations, select up to 10 identities in the list above and choose Check for recommendations. If you implement a provided solution, refresh the table by selecting Check for recommendations to ensure it's no longer listed. If you enable Virtual Deliverability Manager, this table will refresh automatically for all your identities.

Search recommendations

Impact	Identity name	Age	Recommendation/Description	Last checked	Resolve issue
No recommendations found					

Select up to 10 verified domain identities and select **Check for recommendations** on the Identities table above.

[Enable Virtual Deliverability Manager to generate recommendations automatically](#)

Figure 26: Test email sent successfully (this send used the mailbox simulator address)

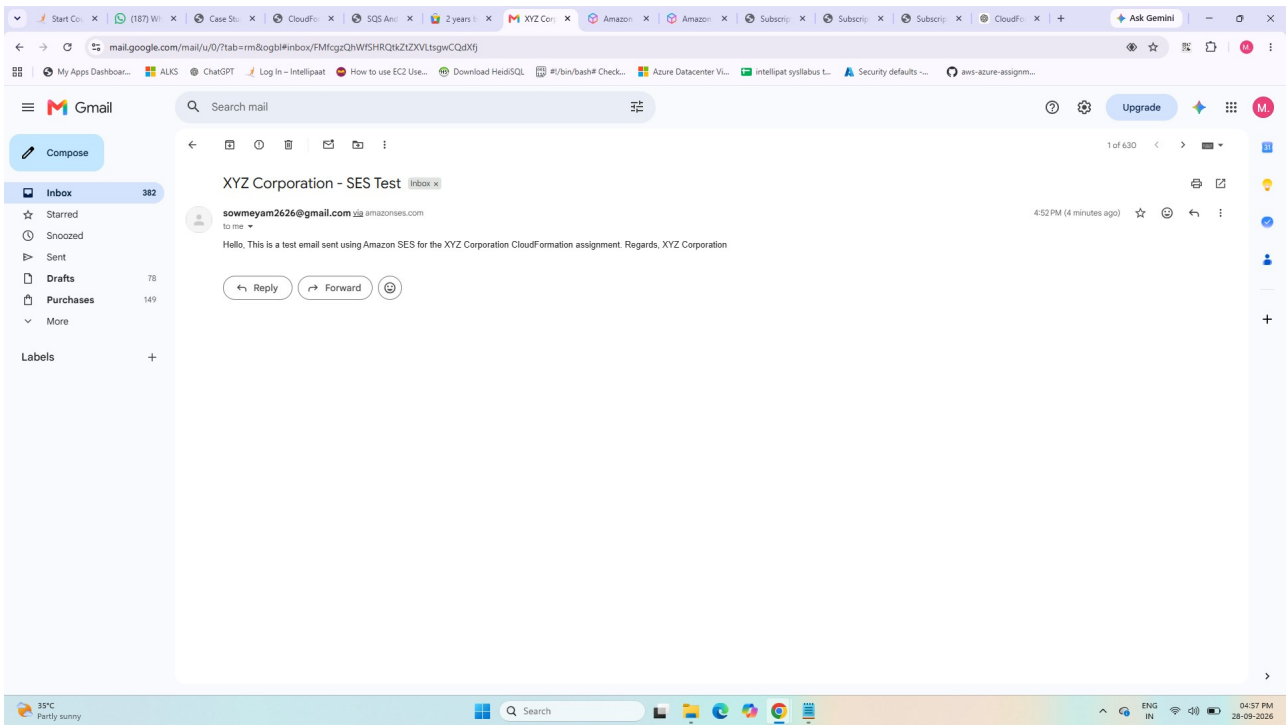


Figure 27: Test email received in the Gmail inbox via amazonses.com

Sandbox note: New SES accounts are in the sandbox, so you can only send to verified addresses. Sending to yourself works because your own address is verified.

Step 8: Review the stacks

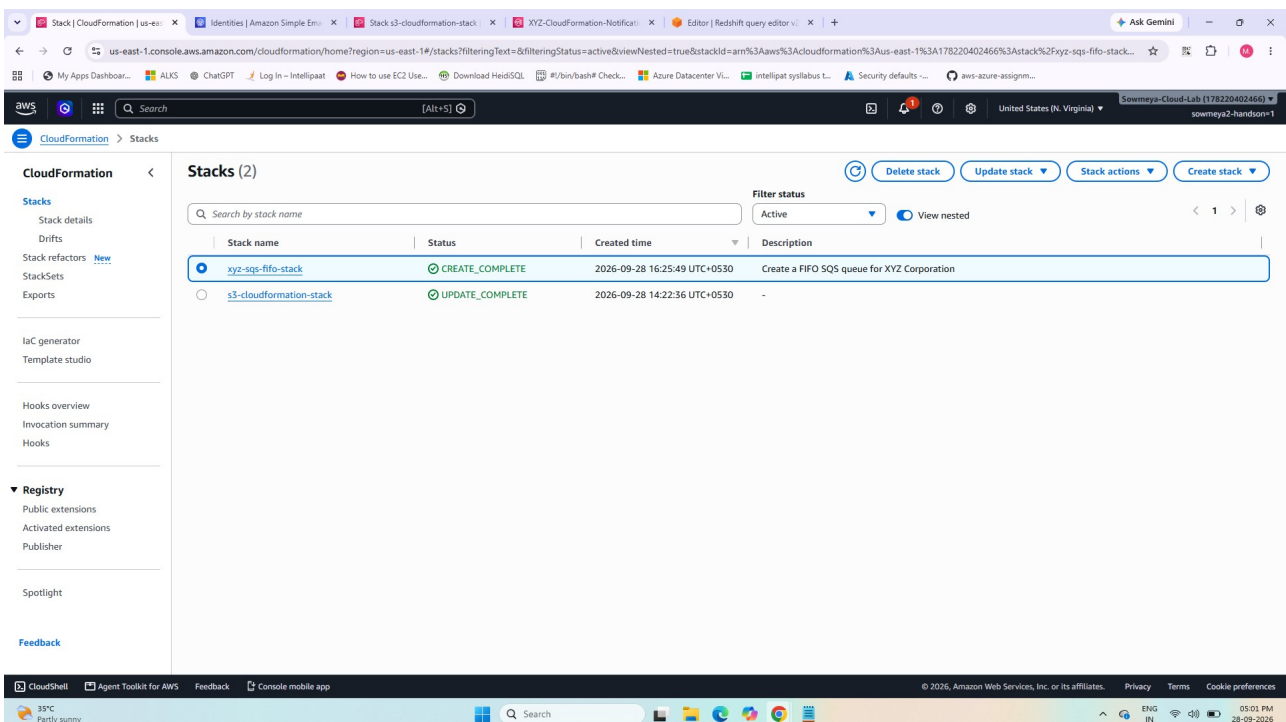


Figure 28: CloudFormation stacks – xyz-sqs-fifo-stack CREATE_COMPLETE and s3-cloudformation-stack UPDATE_COMPLETE

Assignment 3: 3-Tier Web Application with CloudFormation

Problem statement: XYZ Corporation wants to launch a new web application. The code is ready but untested, and the system admins who would normally build the test server are not available.

Tasks:

1. Web tier: launch an instance in a public subnet that allows HTTP and SSH from the internet.
2. Application tier: launch an instance in a private subnet that allows only SSH from the web tier's public subnet.
3. DB tier: launch an RDS MySQL instance in a private subnet that allows port 3306 only from the application tier's private subnet.
4. Set up a Route 53 hosted zone and direct traffic to the EC2 instance.

Also propose a solution so that:

1. The development team can test their code without involving the system admins.
2. RDS DB instances are not deleted when the development team deletes the stack.

Architecture

```
XYZ-Web-VPC 10.0.0.0/16
Web tier public subnet -> XYZ-WebServer (HTTP 80, SSH 22 from internet)
App tier private subnet -> XYZ-AppServer (SSH 22 only from web subnet)
DB tier private subnets -> xyz-rds-mysql (MySQL 3306 only from app subnet)
Route 53 xyzcorporation.internal www -> web server
```

Step 1: Build the web tier template

The template starts with a key pair parameter, the VPC, an Internet Gateway and a public subnet:

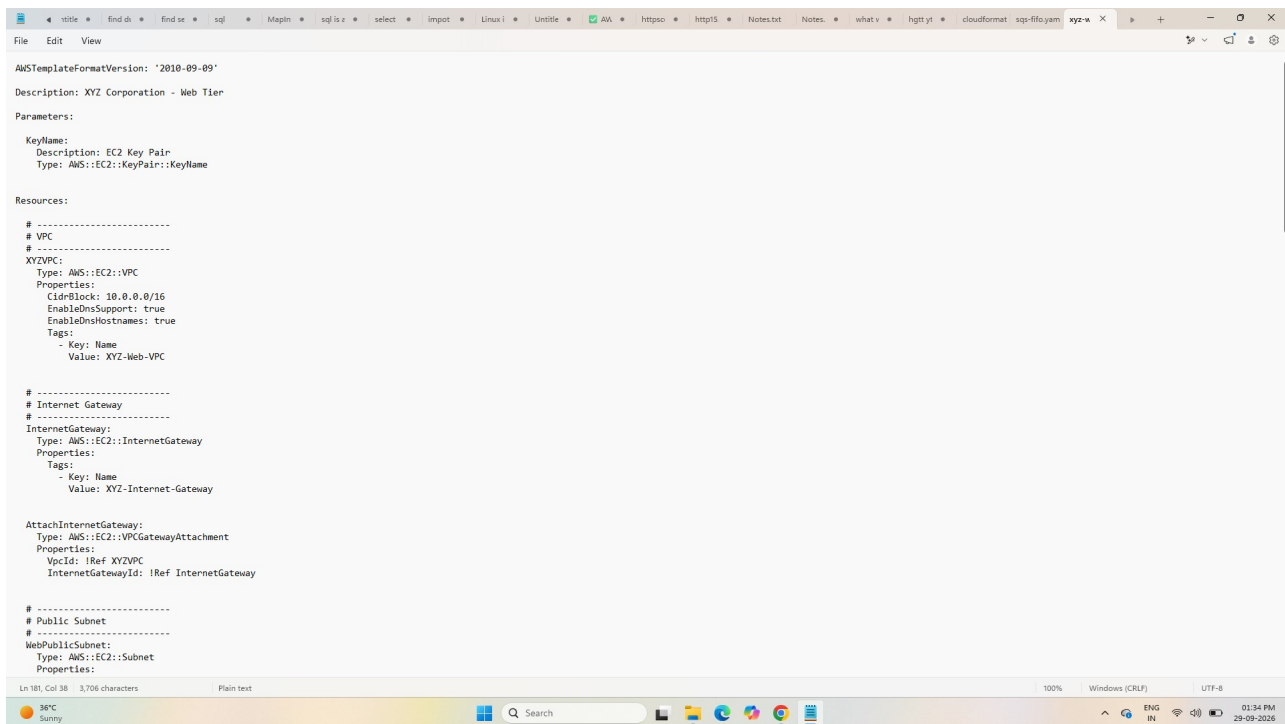


Figure 29: Template – KeyName parameter, XYZVPC (10.0.0.0/16), Internet Gateway and public subnet

The web server installs Apache with user data so it serves a test page:

```

WebServer:
  Type: AWS::EC2::Instance
  Properties:
    ImageId: '{{resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-
kernel-default-x86_64}}'
    InstanceType: t3.micro
    KeyName: !Ref KeyName
    SubnetId: !Ref WebPublicSubnet
    SecurityGroupIds:
      - !Ref WebSecurityGroup
    Tags:
      - Key: Name
        Value: XYZ-WebServer
    UserData:
      Fn::Base64: |
        #!/bin/bash
        dnf update -y
        dnf install -y httpd
        systemctl enable httpd
        systemctl start httpd
        echo "<h1>XYZ Corporation Web Server</h1>" > /var/www/html/index.html

```

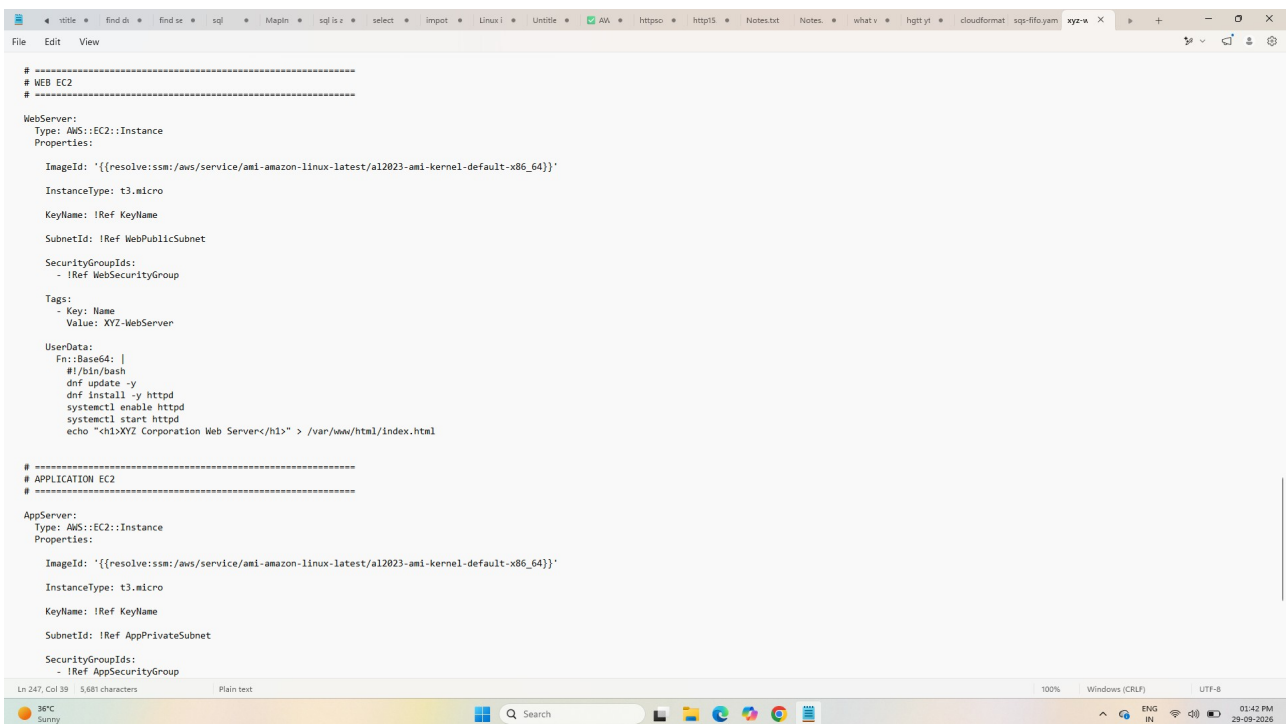


Figure 30: Template – WebServer instance with Apache user data, start of AppServer

Step 2: Create the XYZ-Web-Tier stack

1. **CloudFormation** → **Create stack** → **Upload a template file**.
2. Stack name: **XYZ-Web-Tier**. Parameter **KeyName**: choose your EC2 key pair.
3. Click **Next** → **Submit** and wait for **CREATE_COMPLETE**.

Stacks (1)

XYZ-Web-Tier

Stack info | **Events** | Resources | Outputs | Parameters | Template | Changesets | Git sync

Events (34)

Operation ID	Timestamp	Logical ID	Status	Detailed status	Status reason
819e6770-bbdb-11f1-b769-0eecdfeafe7	2026-09-29 13:28:07 UTC+0530	XYZ-Web-Tier	CREATE_COMPLETE	-	-
819e6770-bbdb-11f1-b769-0eecdfeafe7	2026-09-29 13:28:50 UTC+0530	WebServer	CREATE_COMPLETE	-	-
819e6770-bbdb-11f1-b769-0eecdfeafe7	2026-09-29 13:28:50 UTC+0530	XYZ-Web-Tier	CREATE_IN_PROGRESS	CONFIGURATION_COMPLETE	Eventual consistency check initiated
819e6770-bbdb-11f1-b769-0eecdfeafe7	2026-09-29 13:28:50 UTC+0530	WebServer	CREATE_IN_PROGRESS	CONFIGURATION_COMPLETE	Eventual consistency check initiated

Figure 31: XYZ-Web-Tier events – WebServer and stack CREATE_COMPLETE

Instances (1)

Filter Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4	Elastic IP	IPv6
XYZ-WebServer	i-081437afdb0095221	Running	t3.micro	3/3 checks passed	us-east-1a	ec2-3-239-215-61.com...	3.239.215.61	-	-

All instances

Instance alarm states

In alarm: None. Show instances in alarm.

Service health

Loading service health.

Health events affecting EC2 instances

Loading events...

Figure 32: XYZ-WebServer running in us-east-1a with public IP 3.239.215.61

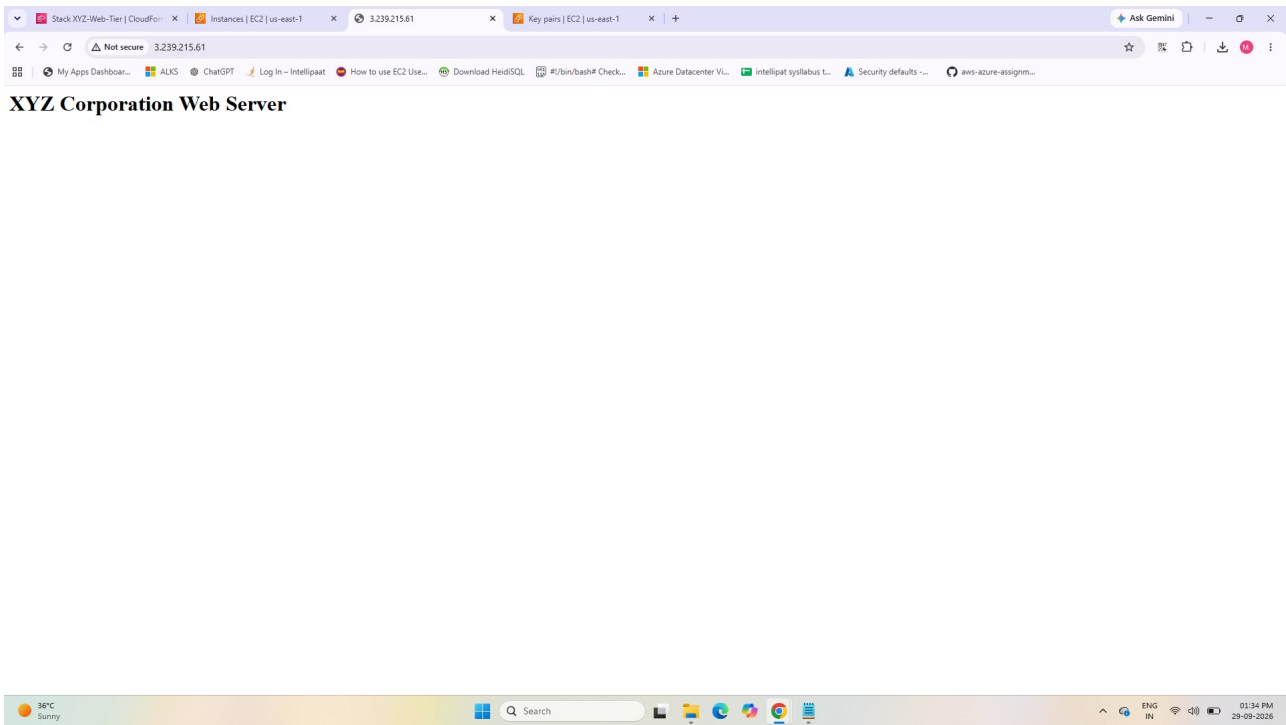


Figure 33: Browsing to the public IP shows the XYZ Corporation Web Server page

Step 3: Add the application tier

1. Add **AppPrivateSubnet**, **AppSecurityGroup** and **AppServer** (t3.micro, no public IP) to the template.
2. Select the stack → **Update stack** → **Make a direct update** → **Replace existing template** and upload the new file.

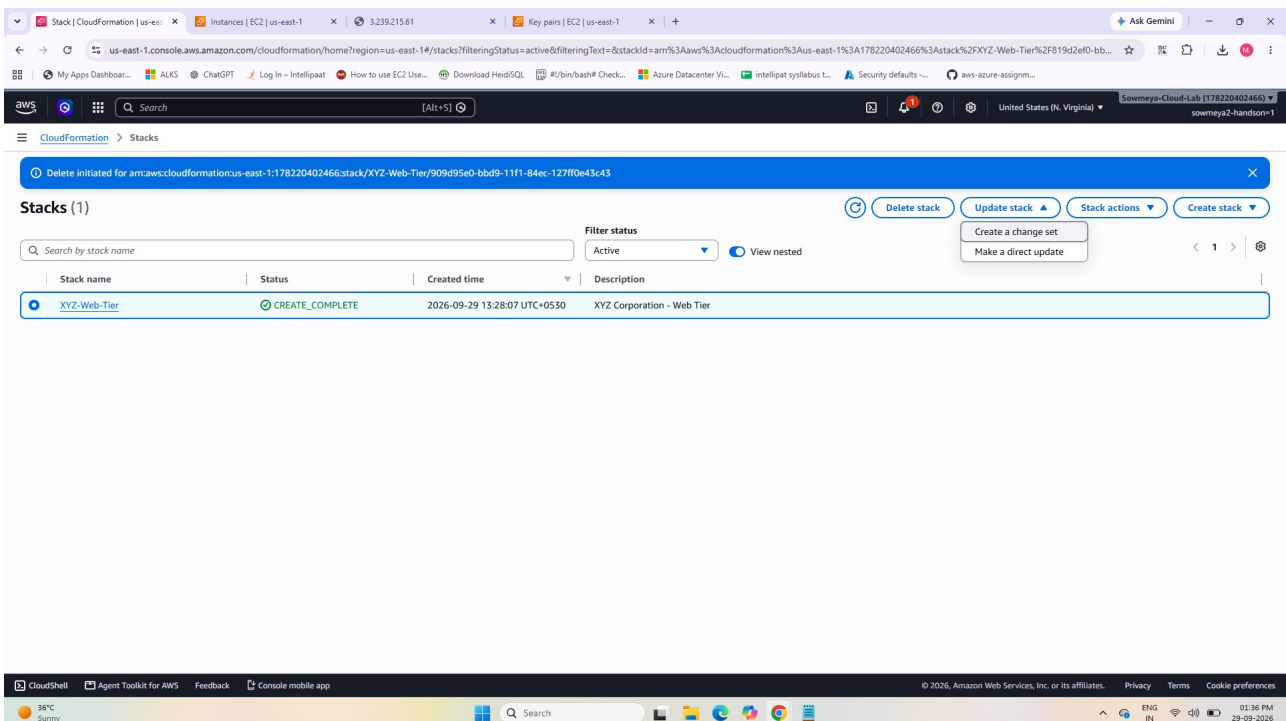


Figure 34: Update stack options – Create a change set or Make a direct update

Stacks (1)

XYZ-Web-Tier
2026-09-29 13:28:07 UTC+0530
UPDATE_COMPLETE

Events (49)

Operation ID	Timestamp	Logical ID	Status	Detailed status	Status reason
15f76f60-bbdd-11f1-b2c1-0e227739bef9	-	-	-	-	-
15f76f60-bbdd-11f1-b2c1-0e227739bef9	2026-09-29 13:40:08 UTC+0530	XYZ-Web-Tier	UPDATE_COMPLETE	-	-
15f76f60-bbdd-11f1-b2c1-0e227739bef9	2026-09-29 13:40:07 UTC+0530	XYZ-Web-Tier	UPDATE_COMPLETE_CLEANUP_IN_PROGRESS	-	-
15f76f60-bbdd-11f1-b2c1-0e227739bef9	2026-09-29 13:40:04 UTC+0530	WebServer	UPDATE_COMPLETE	-	-
15f76f60-bbdd-11f1-b2c1-0e227739bef9	2026-09-29 13:39:54 UTC+0530	AppServer	CREATE_COMPLETE	-	-
15f76f60-bbdd-11f1-b2c1-0e227739bef9	2026-09-29 13:39:42 UTC+0530	AppServer	CREATE_IN_PROGRESS	-	Resource creation Initiated
15f76f60-bbdd-11f1-b2c1-0e227739bef9	2026-09-29 13:39:40 UTC+0530	AppServer	CREATE_IN_PROGRESS	-	-

Figure 35: Update events – AppServer CREATE_COMPLETE and stack UPDATE_COMPLETE

Stacks (1)

XYZ-Web-Tier
2026-09-29 13:28:07 UTC+0530
UPDATE_COMPLETE

Events (49)

Operation ID	Timestamp	Logical ID	Status	Detailed status	Status reason
15f76f60-bbdd-11f1-b2c1-0e227739bef9	-	-	-	-	-
15f76f60-bbdd-11f1-b2c1-0e227739bef9	2026-09-29 13:40:08 UTC+0530	XYZ-Web-Tier	UPDATE_COMPLETE	-	-
15f76f60-bbdd-11f1-b2c1-0e227739bef9	2026-09-29 13:40:07 UTC+0530	XYZ-Web-Tier	UPDATE_COMPLETE_CLEANUP_IN_PROGRESS	-	-
15f76f60-bbdd-11f1-b2c1-0e227739bef9	2026-09-29 13:40:04 UTC+0530	WebServer	UPDATE_COMPLETE	-	-
15f76f60-bbdd-11f1-b2c1-0e227739bef9	2026-09-29 13:39:54 UTC+0530	AppServer	CREATE_COMPLETE	-	-
15f76f60-bbdd-11f1-b2c1-0e227739bef9	2026-09-29 13:39:42 UTC+0530	AppServer	CREATE_IN_PROGRESS	-	Resource creation Initiated
15f76f60-bbdd-11f1-b2c1-0e227739bef9	2026-09-29 13:39:40 UTC+0530	AppServer	CREATE_IN_PROGRESS	-	-
15f76f60-bbdd-11f1-b2c1-0e227739bef9	2026-09-29 13:39:39 UTC+0530	AppSecurityGroup	CREATE_COMPLETE	-	-
15f76f60-bbdd-11f1-b2c1-0e227739bef9	2026-09-29 13:39:34 UTC+0530	AppPrivateSubnet	CREATE_COMPLETE	-	-

Figure 36: Update events – AppPrivateSubnet and AppSecurityGroup created

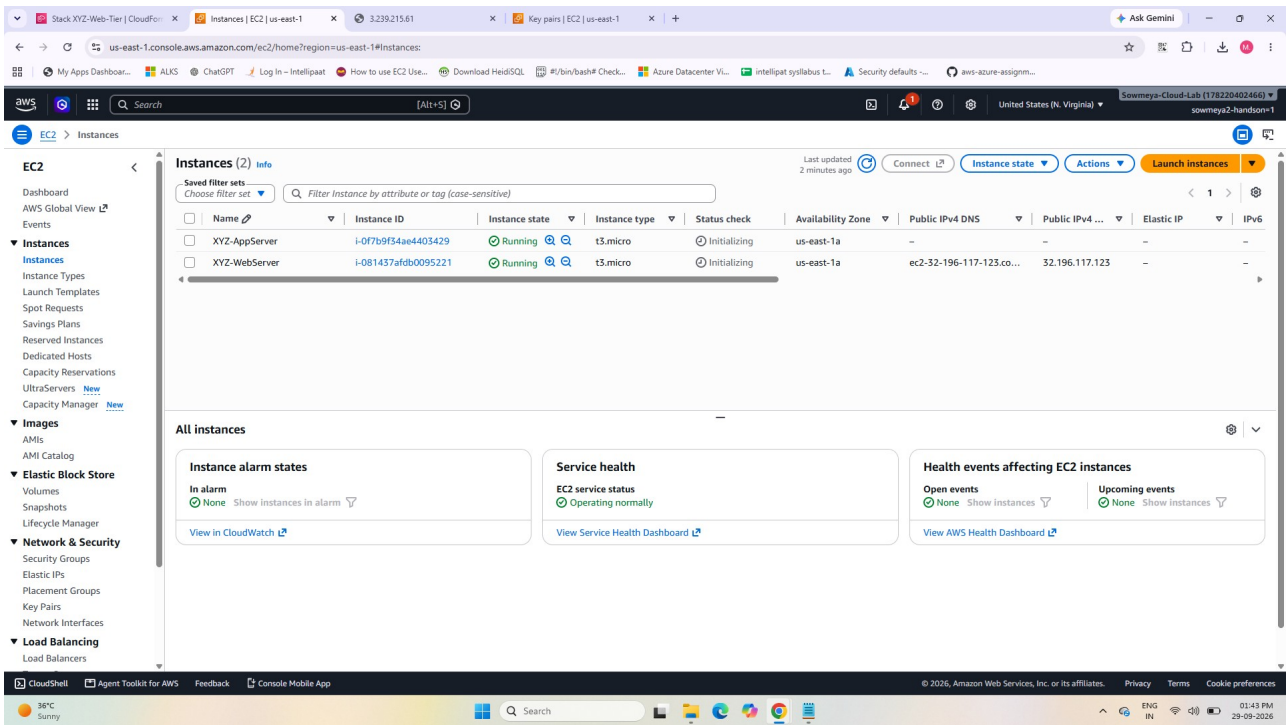


Figure 37: EC2 instances – XYZ-AppServer and XYZ-WebServer

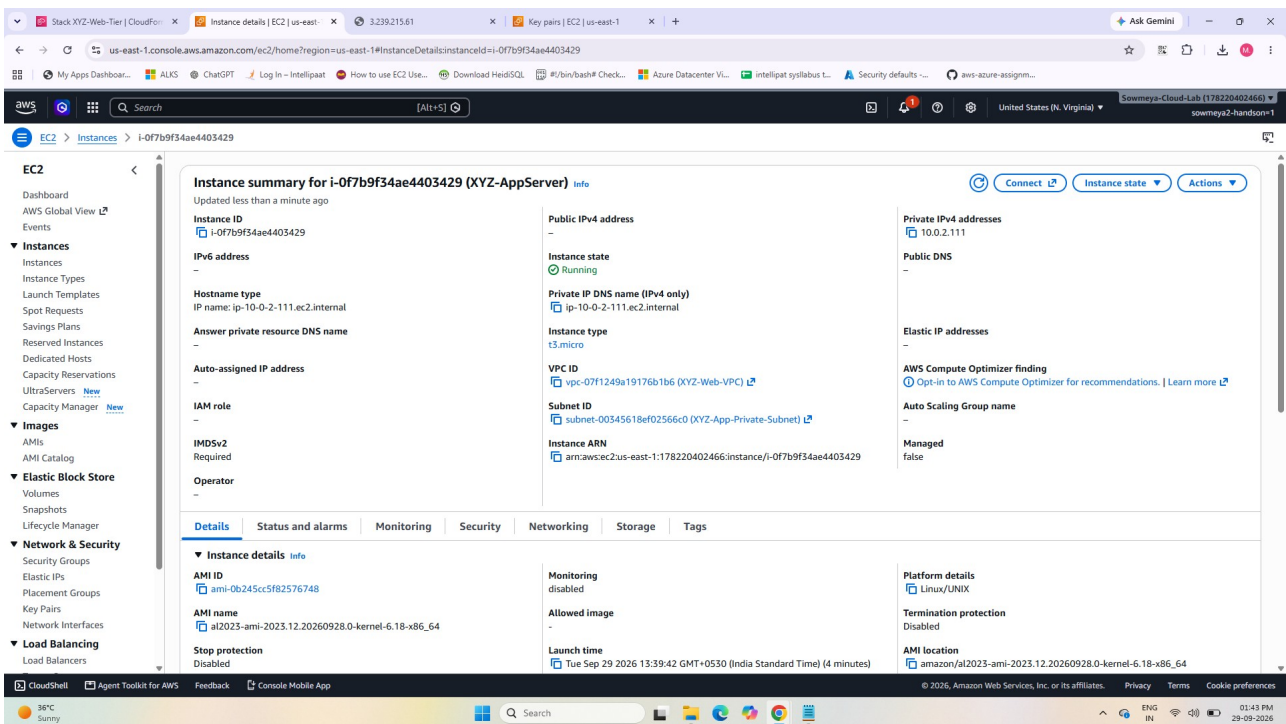


Figure 38: XYZ-AppServer – private IP 10.0.2.111 in XYZ-App-Private-Subnet, no public IP

Step 4: Add the database tier with RDS MySQL

1. Add two DB private subnets (in different AZs), **DBSecurityGroup**, **DBSubnetGroup** and the **XYZDatabase** RDS instance.
2. Add parameters **DBUsername** and **DBPassword** for the master credentials (set **NoEcho**: **true** on the password).
3. Update the stack with a **change set**, review the 7 changes, then click **Submit**.

```

XYZDatabase:
  Type: AWS::RDS::DBInstance
  DeletionPolicy: Retain
  UpdateReplacePolicy: Retain
  Properties:
    DBInstanceIdentifier: xyz-rds-mysql
    Engine: mysql
    EngineVersion: '8.0'
    DBInstanceClass: db.t3.micro
    AllocatedStorage: 20
    StorageType: gp3
    StorageEncrypted: true
    MasterUsername: !Ref DBUsername
    MasterUserPassword: !Ref DBPassword
    DBName: xyzdb
    DBSubnetGroupName: !Ref DBSubnetGroup
    VPCSecurityGroups:
      - !Ref DBSecurityGroup
    PubliclyAccessible: false
    BackupRetentionPeriod: 1

```

```

AppServer:
  Type: AWS::EC2::Instance
  Properties:
    ImageId: '([resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64])'
    InstanceType: t3.micro
    KeyName: !Ref KeyName
    SubnetId: !Ref AppPrivateSubnet
    SecurityGroupIds:
      - !Ref AppSecurityGroup
    Tags:
      - Key: Name
        Value: XYZ-AppServer

# ----- DB Subnet Group -----
DBSubnetGroup:
  Type: AWS::RDS::DBSubnetGroup
  Properties:
    DBSubnetGroupDescription: XYZ Corporation DB Subnet Group
    SubnetIds:
      - !Ref DBPrivateSubnet1
      - !Ref DBPrivateSubnet2
    Tags:
      - Key: Name
        Value: XYZ-DB-Subnet-Group

# ----- RDS MySQL -----
XYZDatabase:
  Type: AWS::RDS::DBInstance
  DeletionPolicy: Retain
  UpdateReplacePolicy: Retain
  Properties:
    DBInstanceIdentifier: xyz-rds-mysql
    Engine: mysql
    EngineVersion: '8.0'
    DBInstanceClass: db.t3.micro
    AllocatedStorage: 20
    StorageType: gp3
    StorageEncrypted: true
    MasterUsername: !Ref DBUsername
    MasterUserPassword: !Ref DBPassword
    DBName: xyzdb
    DBSubnetGroupName: !Ref DBSubnetGroup

```

Figure 39: Template – AppServer, DBSubnetGroup and start of the RDS MySQL resource

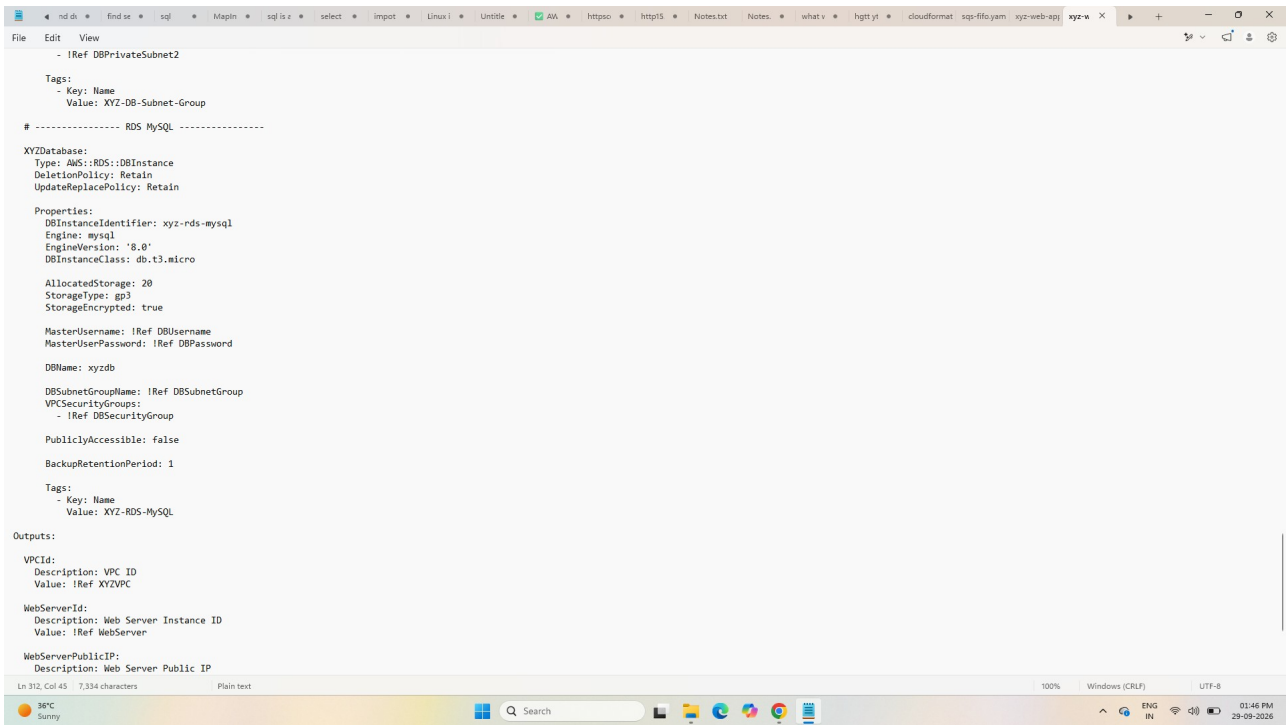


Figure 40: Template – XYZDatabase with DeletionPolicy: Retain, and stack outputs

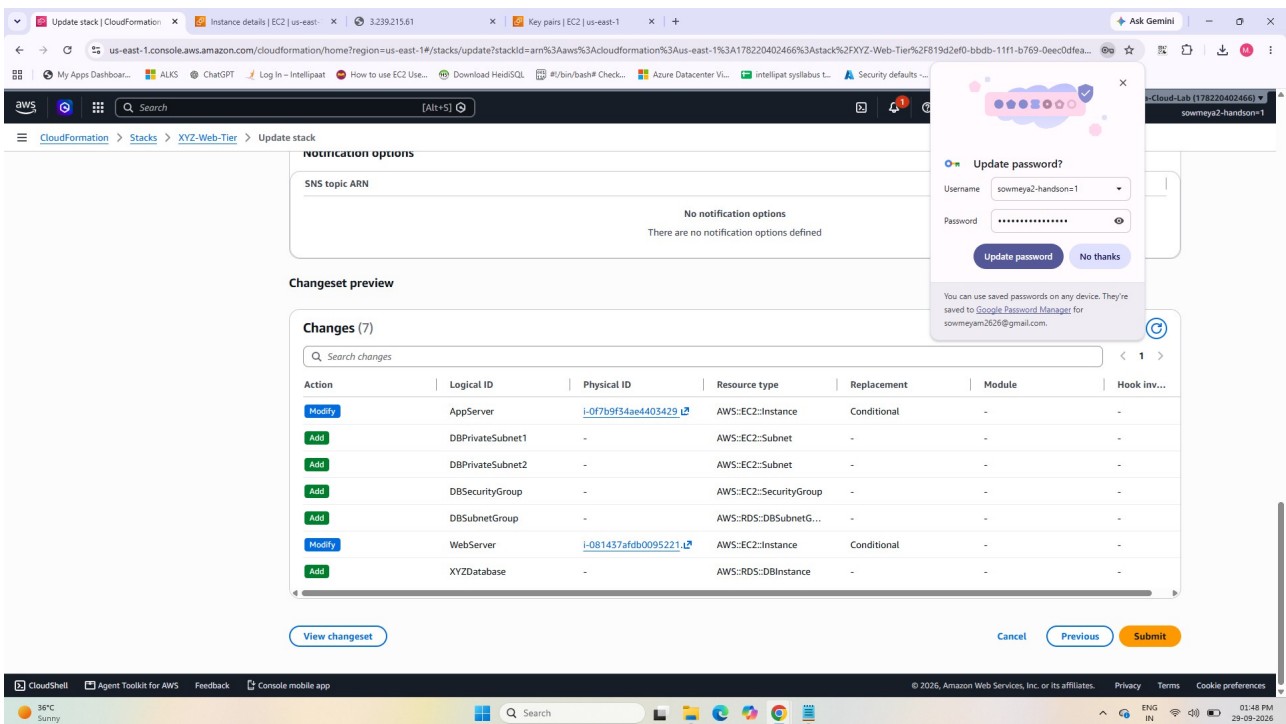


Figure 41: Change set preview – 7 changes (DB subnets, security group, subnet group, RDS instance)

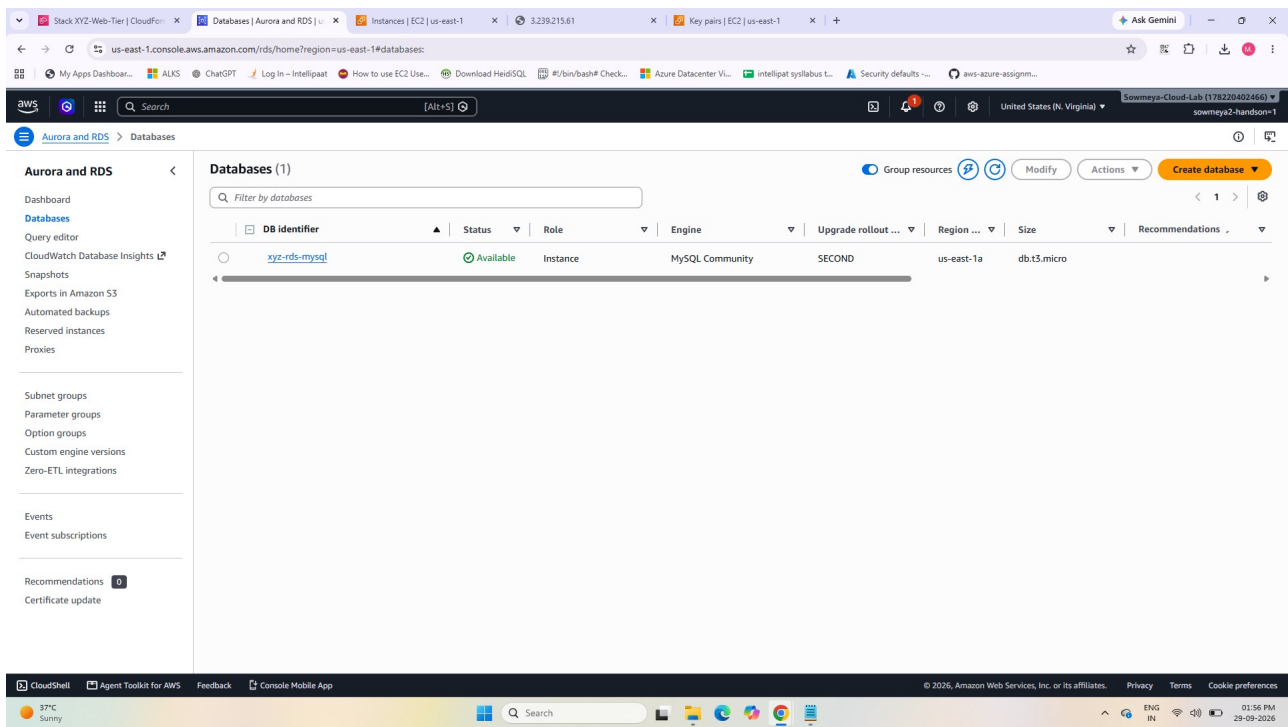


Figure 42: xyz-rds-mysql Available (MySQL Community, db.t3.micro, us-east-1a)

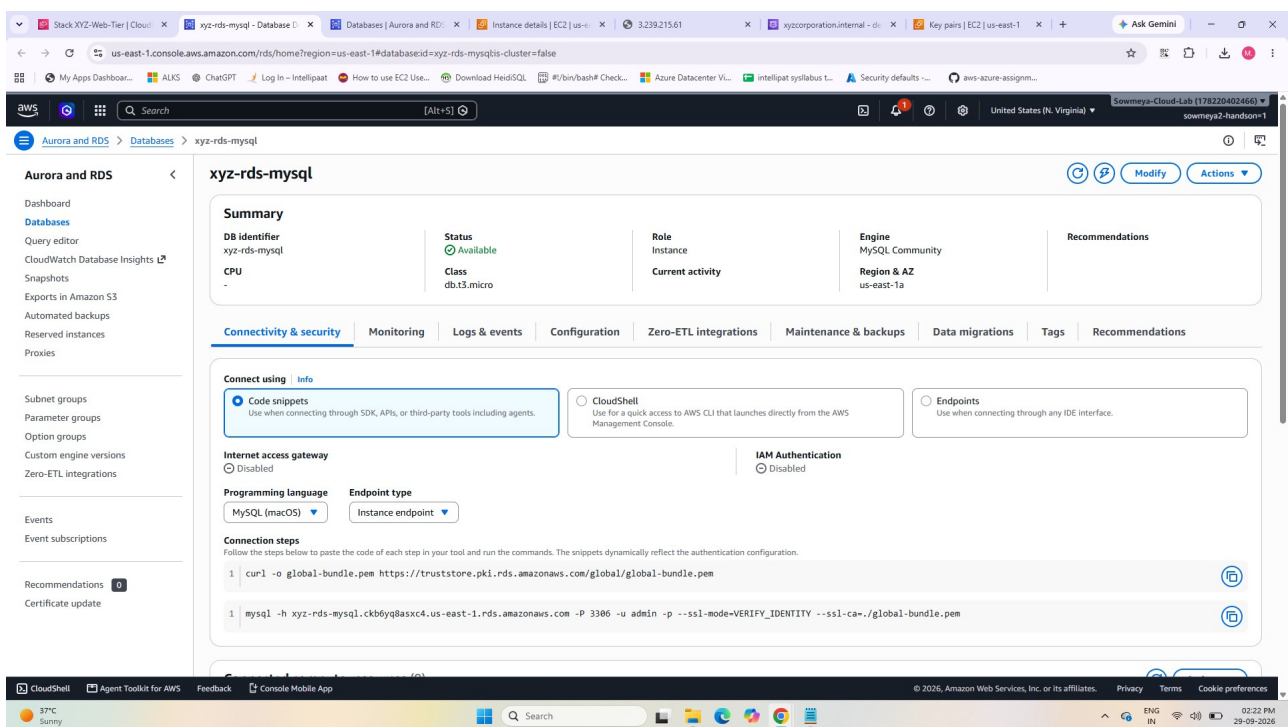


Figure 43: xyz-rds-mysql connectivity details and endpoint

Step 5: Security group rules for each tier

To meet the task, the security groups in the template need these inbound rules:

Security group	Inbound rule	Source
WebSecurityGroup	HTTP (TCP 80) and SSH (TCP 22)	0.0.0.0/0 (internet)
AppSecurityGroup	SSH (TCP 22) only	Web tier public subnet CIDR only
DBSecurityGroup	MySQL (TCP 3306) only	App tier private subnet CIDR only

Step 6: Set up the Route 53 hosted zone

1. Open **Route 53** → **Hosted zones** → **Create hosted zone**.
2. Domain name: **xyzcorporation.internal**. Type: **Private hosted zone**. Region **us-east-1**, VPC **XYZ-Web-VPC**. Create.
3. Click **Create record**: name **www**, type **A**, value **10.0.1.32** (the web server's private IP), TTL 300. Create records.

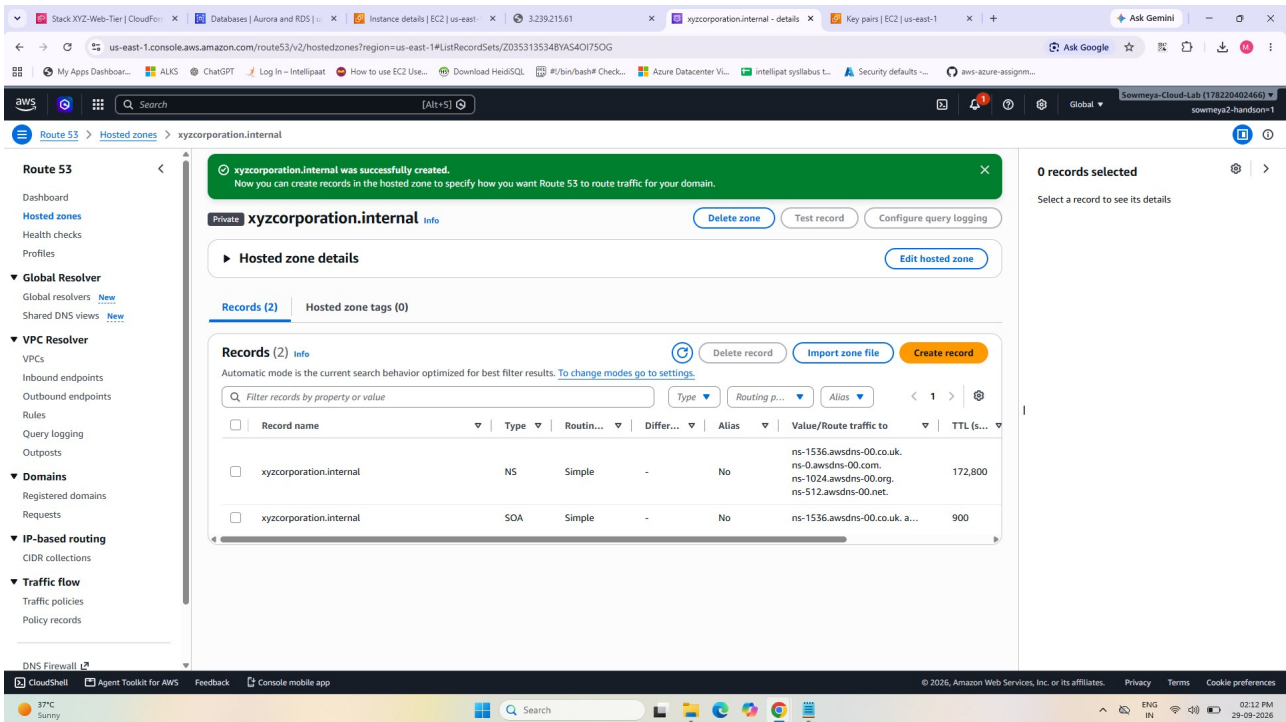


Figure 44: Private hosted zone xyzcorporation.internal created

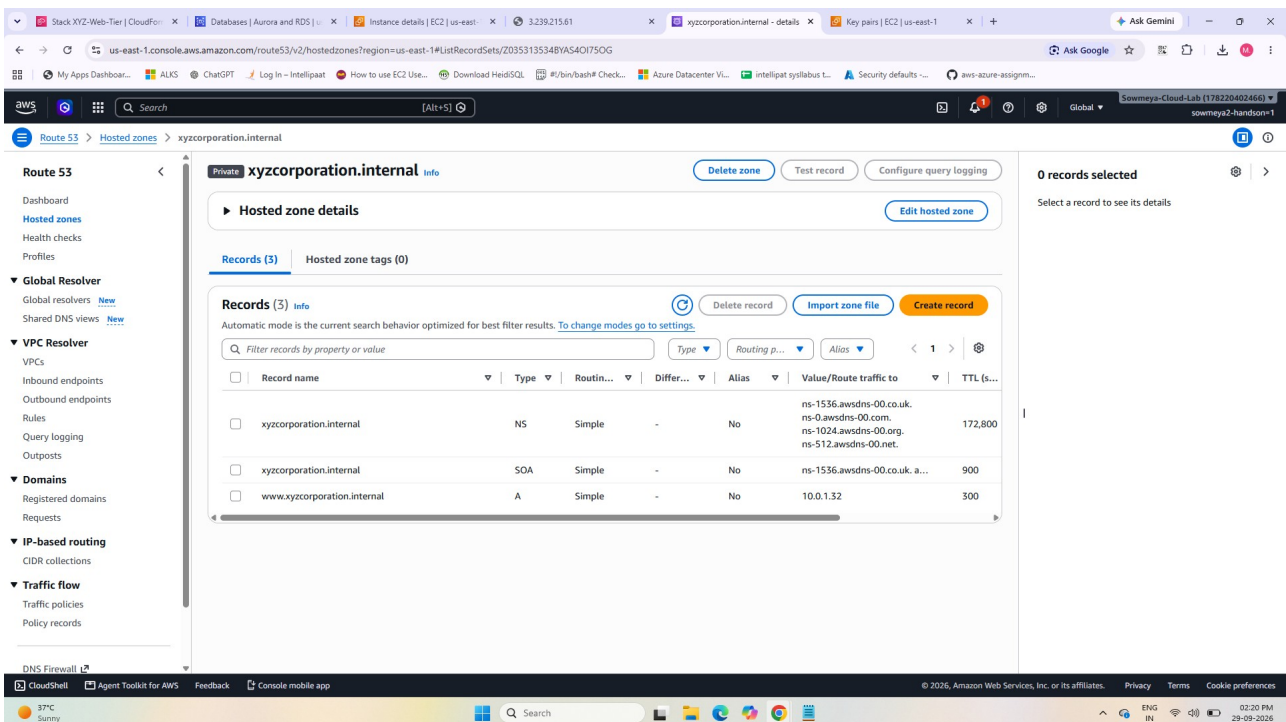


Figure 45: A record www.xyzcorporation.internal pointing to 10.0.1.32

How to test: A private hosted zone resolves only inside the VPC. From an instance in XYZ-Web-VPC, run `curl http://www.xyzcorporation.internal` to reach the web server by name.

Step 7: Proposed solution for the development team

1. Self-service testing with CloudFormation

- The whole 3-tier environment is described in one template, so the development team can create, update and delete it themselves from the CloudFormation console without the system admins.
- Parameters (key pair, DB user and password) let the same template be reused for testing, development and production.
- Change sets show exactly what will change before an update is applied, so updates are safe and repeatable.

2. Keep the RDS database when the stack is deleted

- **DeletionPolicy: Retain** on **XYZDatabase** tells CloudFormation to leave the RDS instance in place when the stack is deleted.
- **UpdateReplacePolicy: Retain** keeps the old instance if an update ever requires the database to be replaced.
- Alternative: **DeletionPolicy: Snapshot** deletes the instance but keeps a final snapshot that can be restored later.

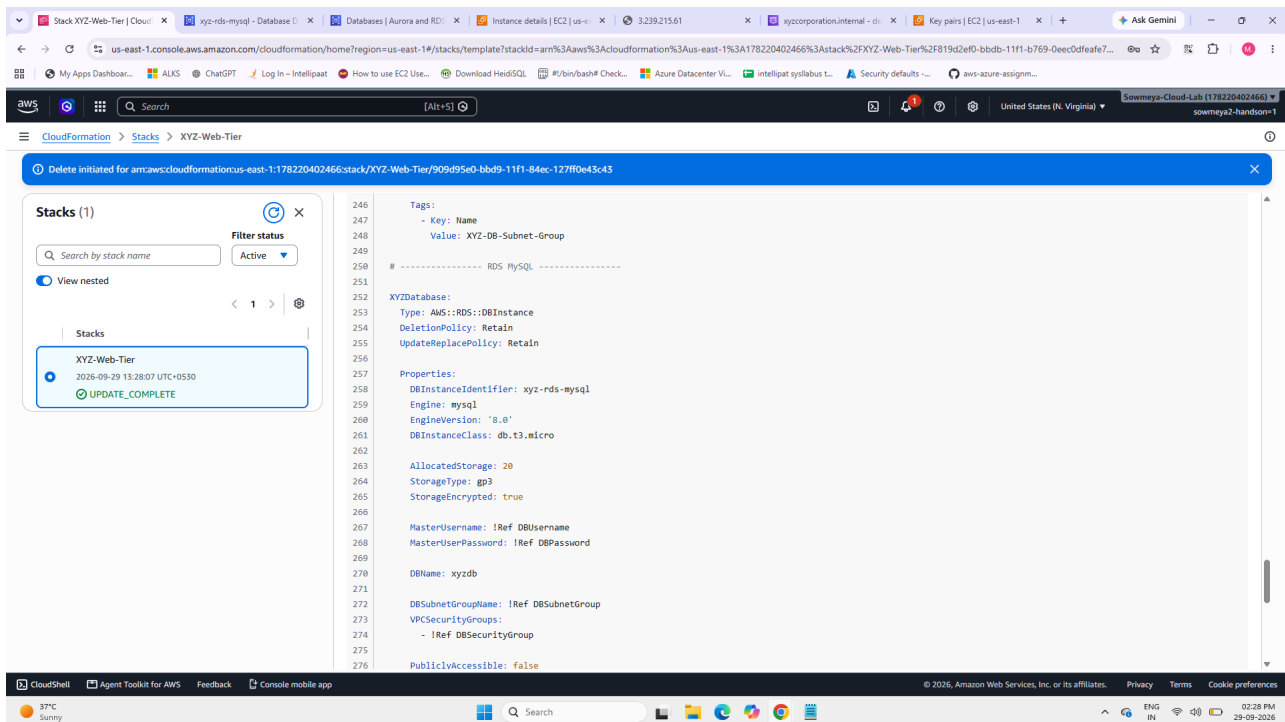


Figure 46: Template in the CloudFormation console – XYZDatabase with DeletionPolicy and UpdateReplacePolicy set to Retain

Conclusion

Each task maps to the configuration and evidence below:

Assignment / task	How it was met	Evidence
1.1 Template from task 1	s3-cloudformation-stack (versioned S3 bucket)	Figures 1, 2
1.2 SNS notifications	SNS topic + confirmed email subscription set in the stack's notification options	Figures 3–11
2.1 FIFO SQS queue	xyz-sqs-fifo-stack creates xyz-test-queue.fifo; message sent and received	Figures 12–18
2.2 SES test email	Email identity verified and test email received	Figures 19, 23, 25, 27
3.1 Web tier	XYZ-WebServer in public subnet serving HTTP	Figures 31–33
3.2 Application tier	XYZ-AppServer in private subnet, no public IP	Figures 37, 38
3.3 DB tier	xyz-rds-mysql in private DB subnets, not publicly accessible	Figures 41–43
3.4 Route 53	Private hosted zone xyzcorporation.internal with www A record	Figures 44, 45
Proposal 1 – self-service	Single CloudFormation template the dev team deploys themselves	Step 7 (Assignment 3)
Proposal 2 – keep RDS	DeletionPolicy and UpdateReplacePolicy set to Retain	Figures 40, 46